

✓ Assignment 1: Evaluating Language Technologies

This assignment must be completed in order to be considered for enrollment in the class. If you are already enrolled, failure to complete this assignment will result in your removal from the course.

Introduction

In this assignment, we will explore different tasks core to the field of NLP. You will implement different evaluation metrics for these tasks and perform analysis of different language technologies.

Learning objectives:

- Understand the breadth of tasks core to the field of NLP.
- Implement various evaluation methods for core NLP tasks.
- Become familiar with prompting existing LLMs via API calls.
- Identify the limitations of automatic evaluation methods.
- Practice using error analysis to identify where language technologies fail and hypothesizing why they might fail.
- Appreciate things we humans do with language that current language models cannot do.

Notes:

- In your solution, keep all code as-is except where it's explicitly mentioned to implement a function.
- Items marked with a star (★) should be answered in markdown text in the notebook. You will include your final notebook as a pdf with your submission.
- Results are written to the `results/` sub-directory as you go, and these are what get autograded.

Submission:

When you are done, run `python3 make_submission.py`. It checks that every file the notebook should have written is present, valid, and actually filled in, then packs them into `submission.zip` for you to upload to gradescope. Two things go in it:

- All the json files saved inside the `results/` directory. These will be autograded.
- A PDF copy of your completed notebook, titled "HW1.pdf". This is used to grade your answers to the problems marked with a (★), so check that your plots rendered before exporting.

Setup: You will need to download ollama <https://ollama.com> and install the following models with the commands:

```
ollama pull qwen3:8b
```

```
ollama pull llama3.2:1b
```

```
ollama pull llama3.2
```

```
ollama pull gemma3:1b
```

```
ollama pull gemma3:4b
```

You will need about 13GB of storage to download these models. Make sure the ollama binary is in [/usr/local/bin/](#) or you know where it is, so you can call it on the command line.

✓ Running on Google Colab

Set **Runtime** → **Change runtime type** → **GPU**, then run the cell below once. It clones the repo, installs everything, points your inference cache at Drive so a disconnect can't throw away work, and starts ollama.

Running on your own machine instead? The cell does nothing — follow the README.

```
# Colab setup. On a local machine this cell is a no-op.
import sys

if "google.colab" in sys.modules:
    import os, subprocess, time

    STUDENT_REPO = "https://github.com/zinengtang/cs183-fall26-a1.git"
    DRIVE_DIR = "/content/drive/MyDrive/cs183-a1"

    # 1. code and data, since opening a notebook from GitHub brings only the notebook
    if not os.path.exists("/content/repo"):
        subprocess.run(["git", "clone", "-q", STUDENT_REPO, "/content/repo"], check=True)
    os.chdir("/content/repo")

    # 2. dependencies (ffmpeg is already on Colab). Capture rather than -q: a bare
    # CalledProcessError tells you nothing about which package refused to install.
    installed = subprocess.run([sys.executable, "-m", "pip", "install", "-r", "requirements.",
                               capture_output=True, text=True])
    if installed.returncode != 0:
        print(installed.stdout[-2000:])
        print(installed.stderr[-2000:])
        raise RuntimeError("pip install failed -- see the output above for which package and

    # 3. keep generations and results on Drive -- the runtime's own disk does not
    # survive a disconnect, and re-running inference costs far more than this does
    from google.colab import drive
    drive.mount("/content/drive")
    for sub in ("inference_cache", "results"):
        os.makedirs(f"{DRIVE_DIR}/{sub}", exist_ok=True)
        subprocess.run(["rm", "-rf", sub], check=True)
        os.symlink(f"{DRIVE_DIR}/{sub}", sub)

    # 4. ollama. Colab has no systemd, so the installer cannot start the server for us.
    # Only the four small models are pulled here. The 5.2GB judge model that Part 4
    # needs is pulled at Part 4, so Parts 1-3 are not stuck behind that download.
    # The installer unpacks a .tar.zst and exits with an error if zstd is missing,
    # which it is on Colab. There is no .tgz build to fall back to any more.
    if subprocess.run(["apt-get", "install", "-y", "-qq", "zstd"]).returncode != 0:
        subprocess.run(["apt-get", "update", "-qq"], check=False)
        subprocess.run(["apt-get", "install", "-y", "-qq", "zstd"], check=True)

    installer = subprocess.run("curl -fsSL https://ollama.com/install.sh | sh",
                               shell=True, capture_output=True, text=True)
    if installer.returncode != 0:
        print(installer.stdout[-2000:])
        print(installer.stderr[-2000:])
        raise RuntimeError("the ollama installer failed, see the output above for why")

    subprocess.Popen(["ollama", "serve"])
    time.sleep(5)
    for model in ["llama3.2:1b", "llama3.2", "gemma3:1b", "gemma3:4b"]:
        print(f"pulling {model}")
        subprocess.run(["ollama", "pull", model], check=True)

    print("\nReady. Run the rest of the notebook from the top.")
else:
    print("Not on Colab -- see the README for local setup.")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/",
pulling llama3.2:1b
pulling llama3.2
pulling gemma3:1b
pulling gemma3:4b
```

Ready. Run the rest of the notebook from the top.

▼ Part 0: Setup

```
import pandas as pd
from pandas.core.frame import DataFrame
import aiohttp
import asyncio
import json
from tqdm import tqdm
from typing import Dict
import os

# ollama serves requests from a small queue, so firing hundreds at once just builds a
# backlog that runs past aiohttp's 5 minute default timeout. Bound both explicitly.
MAX_CONCURRENT_REQUESTS = 8
REQUEST_TIMEOUT_SECONDS = 600

class OllamaError(RuntimeError):
    pass

def save_results(results, filename):
    with open(filename, "w") as f:
        json.dump(results, f, indent=4)

async def run_inference(model_name, prompt, temperature=0.0, max_tokens=16, session=None):
    """
    curl http://localhost:11434/api/generate -d '{
        "model": "llama3.2",
        "prompt": "Why is the sky blue?",
        "stream": false,
    }'
    """
    payload = {
        "model": model_name,
        "prompt": prompt,
        "stream": False,
        # Reasoning models spend their whole token budget thinking before they answer,
        # which for a 16-token judge call means an empty response. Models without a
        # thinking mode ignore this.
        "think": False,
        "options": {
            "temperature": temperature,
            "num_predict": max_tokens,
        },
    },

    async def _post(open_session):
        async with open_session.post("http://localhost:11434/api/generate", json=payload) as r:
            if response.status != 200:
                # e.g. a model that has not been pulled yet answers 404 with an error
                # payload, whose .get("response", "") is an empty string -- silently
                # caching these looks exactly like a model that scores zero
```

```

        # caching those looks exactly like a model that scores zero.
        raise OllamaError(f"ollama returned {response.status}: {(await response.text)}")
    result = await response.json()
    if "error" in result:
        raise OllamaError(result["error"])
    return result.get("response", "")

if session is not None:
    return await _post(session)
timeout = aiohttp.ClientTimeout(total=REQUEST_TIMEOUT_SECONDS)
async with aiohttp.ClientSession(timeout=timeout) as new_session:
    return await _post(new_session)

async def run_batch_inference(model_name, prompts, temperature=0.0, max_tokens=16):
    progress = tqdm(total=len(prompts))
    semaphore = asyncio.Semaphore(MAX_CONCURRENT_REQUESTS)
    failures = []

    timeout = aiohttp.ClientTimeout(total=REQUEST_TIMEOUT_SECONDS)
    async with aiohttp.ClientSession(timeout=timeout) as session:
        async def _run_inference(prompt):
            async with semaphore:
                try:
                    result = await run_inference(model_name, prompt, temperature, max_token)
                except Exception as e:
                    # returning None instead of raising keeps one bad request from
                    # discarding every other result in the batch
                    failures.append(f"{type(e).__name__}: {e}")
                    result = None
                progress.update(1)
                return result
        results = await asyncio.gather(*[_run_inference(prompt) for prompt in prompts])

    if failures:
        print(f"{len(failures)}/{len(prompts)} requests to {model_name} failed and were not
            re-run this cell to retry them. First failure: {failures[0]}")
    return results

def cache_inferences(model_name, exp_name, prompts, inferences):
    # skip failed requests so they are retried rather than cached as missing answers
    fresh = {prompt: inference for prompt, inference in zip(prompts, inferences) if inference is not None}
    if os.path.exists(f"inference_cache/{model_name}_{exp_name}.json"):
        with open(f"inference_cache/{model_name}_{exp_name}.json", "r") as f:
            cache = json.load(f)
            cache.update(fresh)
    else:
        cache = fresh
    with open(f"inference_cache/{model_name}_{exp_name}.json", "w") as f:
        json.dump(cache, f)

def load_inferences(model_name, exp_name, prompts):
    if os.path.exists(f"inference_cache/{model_name}_{exp_name}.json"):
        with open(f"inference_cache/{model_name}_{exp_name}.json", "r") as f:
            cache = json.load(f)
            return [cache[prompt] if prompt in cache else None for prompt in prompts]
    return [None for _ in prompts]

def get_remaining_prompts(model_name, exp_name, prompts):
    if os.path.exists(f"inference_cache/{model_name}_{exp_name}.json"):
        with open(f"inference_cache/{model_name}_{exp_name}.json", "r") as f:
            cache = json.load(f)
            return [prompt for prompt in prompts if prompt not in cache]
    return prompts

```

```
models = [
    "llama3.2:1b" ,
    "llama3.2",
    "gemma3:1b",
    "gemma3:4b",
]
```

✓ Part 1: Text classification (25 points)

Here, we will evaluate different LLMs on their ability to perform newsgroup classification. You will implement four standard metrics for evaluating a classifier and create a visualization of the errors using a confusion matrix. We will be using a classic NLP dataset, 20 Newsgroups (<http://qwone.com/~jason/20Newsgroups/>).

✓ Part 1.0: Setup

Load the data we will be using for evaluation. This is the development set of the dataset, so it's ok for us to do analysis with it.

We will load the possible labels from the training set, rather than the development set. If a new label has to be predicted at test-time, we wouldn't have learned it from training only on the training set!

```
classification_inputs = pd.read_csv('A1_Data/dev_data.csv')
classification_labels = pd.read_csv('A1_Data/dev_labels.csv')
classification_data = pd.merge(classification_inputs, classification_labels, on="id", how="left")

print(f"Loaded newsgroup data of shape {classification_data.shape} with columns {classification_data.columns}")

labels = pd.read_csv('A1_Data/train_labels.csv')['newsgroup'].unique()
label_list = ", ".join(sorted(list(labels)))

print(f"Found {len(labels)} possible target labels: ")
print(label_list)

def normalize_prediction(text):
    """Clean a raw model response up so it can be compared against a label.

    Model output is not guaranteed to be exactly one of the newsgroup names.
    Responses here routinely carry a trailing newline, and a model that ignores
    the "no other text" instruction may wrap the label in something longer.
    Deciding how to read a model's answer is part of the evaluation, not a
    detail -- for one of these models, skipping this step more than halves its
    measured accuracy.
    """
    if text is None:
        # the request failed and was not cached -- re-run the inference cell to retry it
        return ""
    return text.strip()
```

```
Loaded newsgroup data of shape (200, 3) with columns ['id', 'text', 'newsgroup'].
Found 20 possible target labels:
alt.atheism, comp.graphics, comp.os.ms-windows.misc, comp.sys.ibm.pc.hardware, comp.sys.mac.
```

Now let's run inference with the different classifiers. It will take some time to run inference.

If you cannot get ollama running, ([inference_cache_reference/](#)) holds our own generations for Parts 1, 3 and 4.0 — copy them into ([inference_cache/](#)) and the cells below will use those. Say so in your writeup if

you do, and note that Part 4's judge is not cached, so 4.1, 4.2 and 4.4 need a working ollama regardless.

```
def get_newsgroup_prompt(text):
    newsgroup_prompt = "In which of the following newsgroups is the following email most li
    newsgroup_prompt += f"\n\nNewsgroups:\n{label_list}\n\nEmail:\n{text}"
    return newsgroup_prompt

prompts = [get_newsgroup_prompt(text) for text in classification_data["text"]]
for model in models:
    print(f"Running inference for {model}")
    remaining_prompts = get_remaining_prompts(model, "pt1_0_classification_data", prompts)
    curr_inferences = await run_batch_inference(model, remaining_prompts, temperature=0.0,
    cache_inferences(model, "pt1_0_classification_data", remaining_prompts, curr_inferences
    inferences = load_inferences(model, "pt1_0_classification_data", prompts)
    classification_data[f"predictions_{model}"] = inferences
```

```
Running inference for llama3.2:1b
0it [00:00, ?it/s]
Running inference for llama3.2
0it [00:00, ?it/s]
Running inference for gemma3:1b
0it [00:00, ?it/s]
Running inference for gemma3:4b
0it [00:00, ?it/s]
```

classification_data

	id	text	newsgroup	predictions_llama3.2:1b	predic
0	0	From: tim@kimba.catt.citri.edu.au (Tim Liddelo...	comp.windows.x	talk.politics.misc	comp.s
1	1	From: petrack@vnet.IBM.COM\nSubject: disabling...	comp.sys.mac.hardware	talk.politics.misc	
2	2	From: Lars.Jorgensen@p7.syntax.bbs.bad.se (Lar...	comp.graphics	talk.politics.misc	
3	3	From: dnh@mfltd.co.uk (Des Herriott)\nSubject:...	comp.windows.x	talk.politics.misc	
4	4	From: willisw@willisw.ENG.CLEMSON.edu (Bill Wi...	comp.sys.ibm.pc.hardware	talk.politics.guns	
...	...	...	...	...	...
195	195	From: fox@graphics.cs.nyu.edu (David Fox)\nSub...	soc.religion.christian	talk.politics.misc	

Next steps: [New interactive sheet](#)

✓ Part 1.1: Confusion Matrix (5 points)

Complete the following functions. The first one should computes a confusion matrix for a set of predictions, and the second one should use your favorite tool (e.g. Matplotlib) to visualize it as a heatmap.

We're implementing the confusion matrix first because you might find it useful when completing the rest of Part 1.

If the model messes up the output format (e.g. outputs something that isn't one of the labels), the prediction

should not be included in the matrix counts.

```
for model in models:
    col = f"predictions_{model}"
    print(model, "missing:", classification_data[col].isna().sum())
```

```
llama3.2:1b missing: 0
llama3.2 missing: 0
gemma3:1b missing: 0
gemma3:4b missing: 0
```

```
def part1_1_cm(x: DataFrame, y: str) -> Dict[str, Dict[str, int]]:
    label_names = sorted(labels)

    # rows = true labels, columns = predicted labels
    cm = {
        true_label: {pred_label: 0 for pred_label in label_names}
        for true_label in label_names
    }

    prediction_col = f"predictions_{y}"

    for _, row in x.iterrows():
        true_label = row["newsgroup"]
        pred_label = normalize_prediction(row[prediction_col])

        # Assignment says invalid model outputs should be excluded
        # from the confusion matrix.
        if pred_label in label_names:
            cm[true_label][pred_label] += 1

    return cm

def part_1_1_vis(x: Dict[str, Dict[str, int]]):
    import matplotlib.pyplot as plt
    import numpy as np

    label_names = sorted(x.keys())

    matrix = np.array([
        [x[true_label][pred_label] for pred_label in label_names]
        for true_label in label_names
    ])

    fig, ax = plt.subplots(figsize=(12, 10))
    image = ax.imshow(matrix)

    ax.set_xticks(range(len(label_names)))
    ax.set_yticks(range(len(label_names)))

    ax.set_xticklabels(label_names, rotation=90)
    ax.set_yticklabels(label_names)

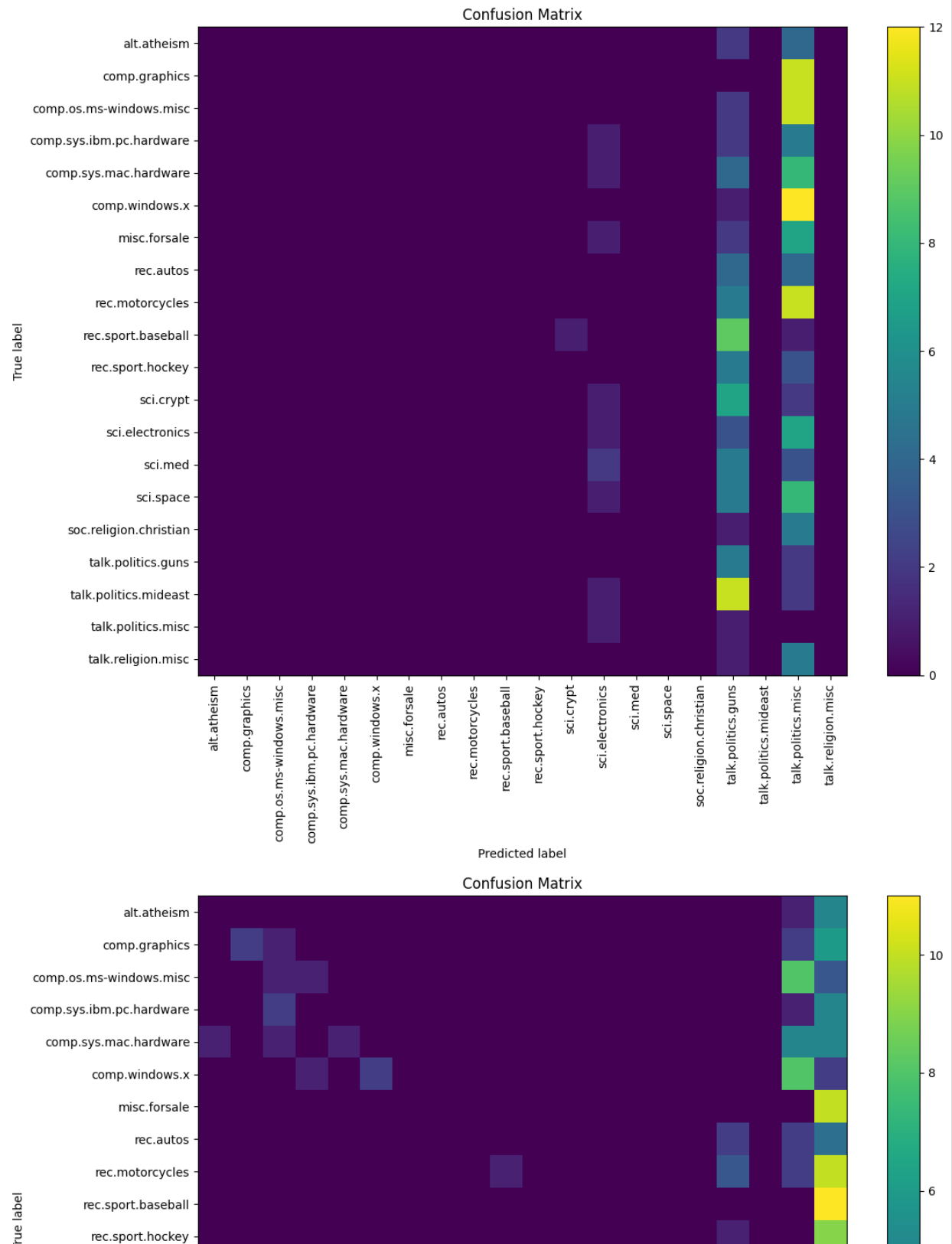
    ax.set_xlabel("Predicted label")
    ax.set_ylabel("True label")
    ax.set_title("Confusion Matrix")

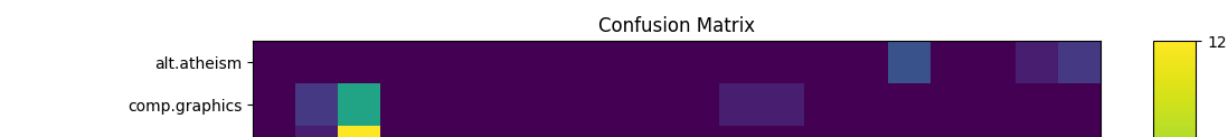
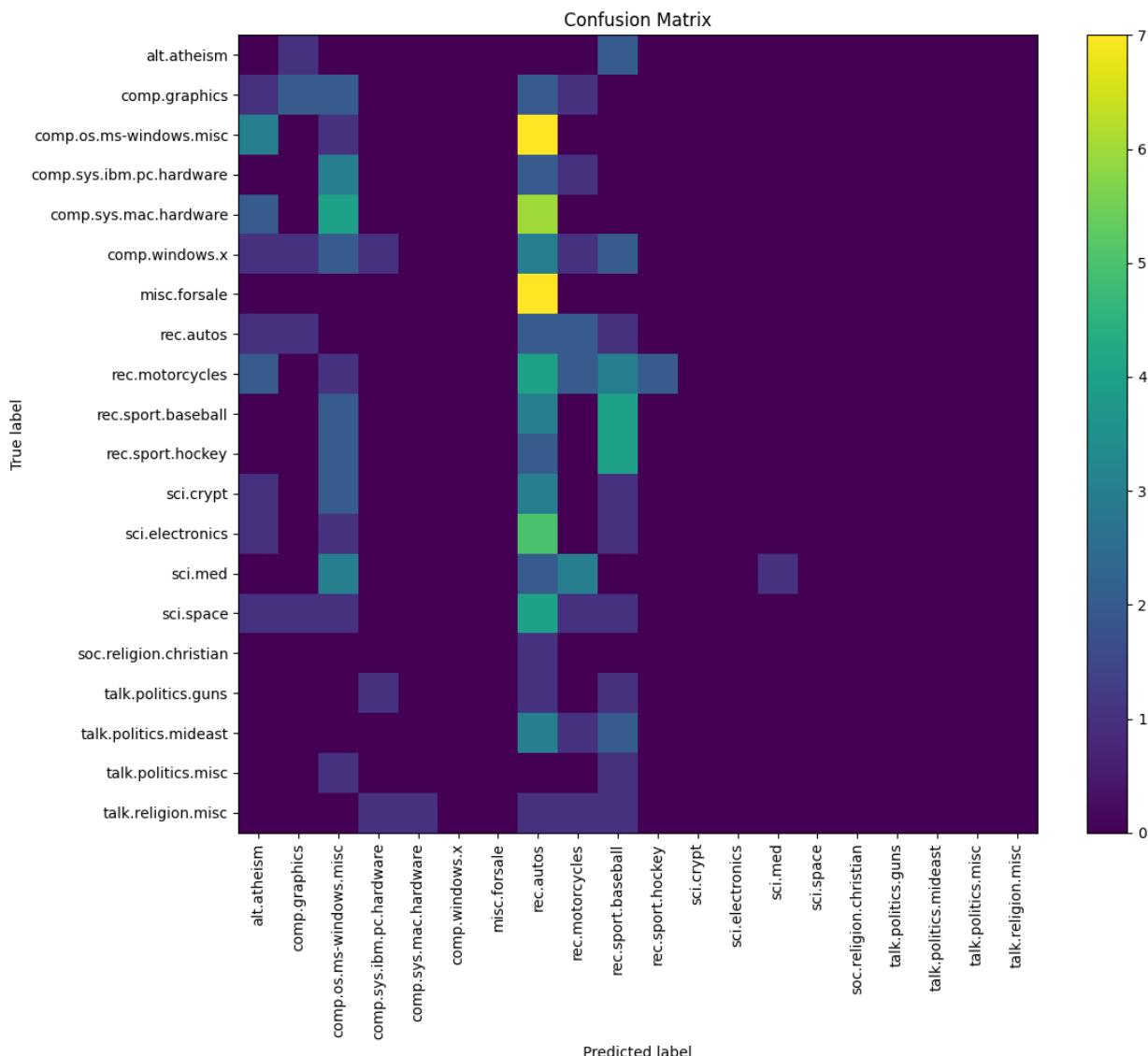
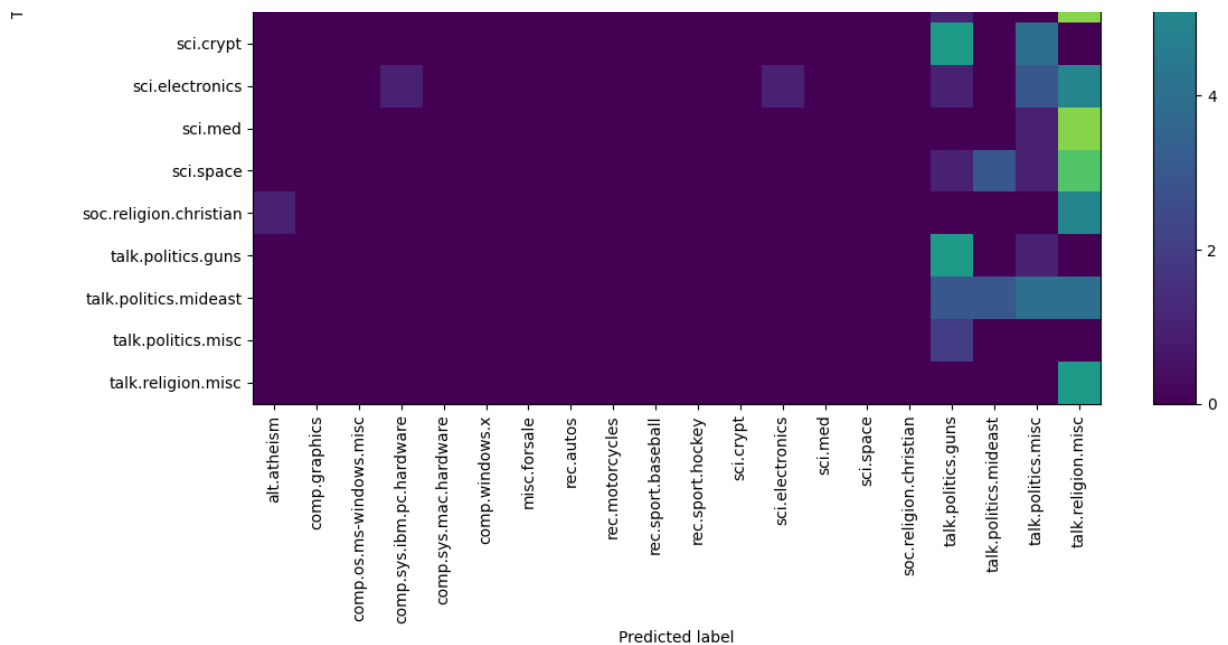
    fig.colorbar(image, ax=ax)
    plt.tight_layout()
    plt.show()
```

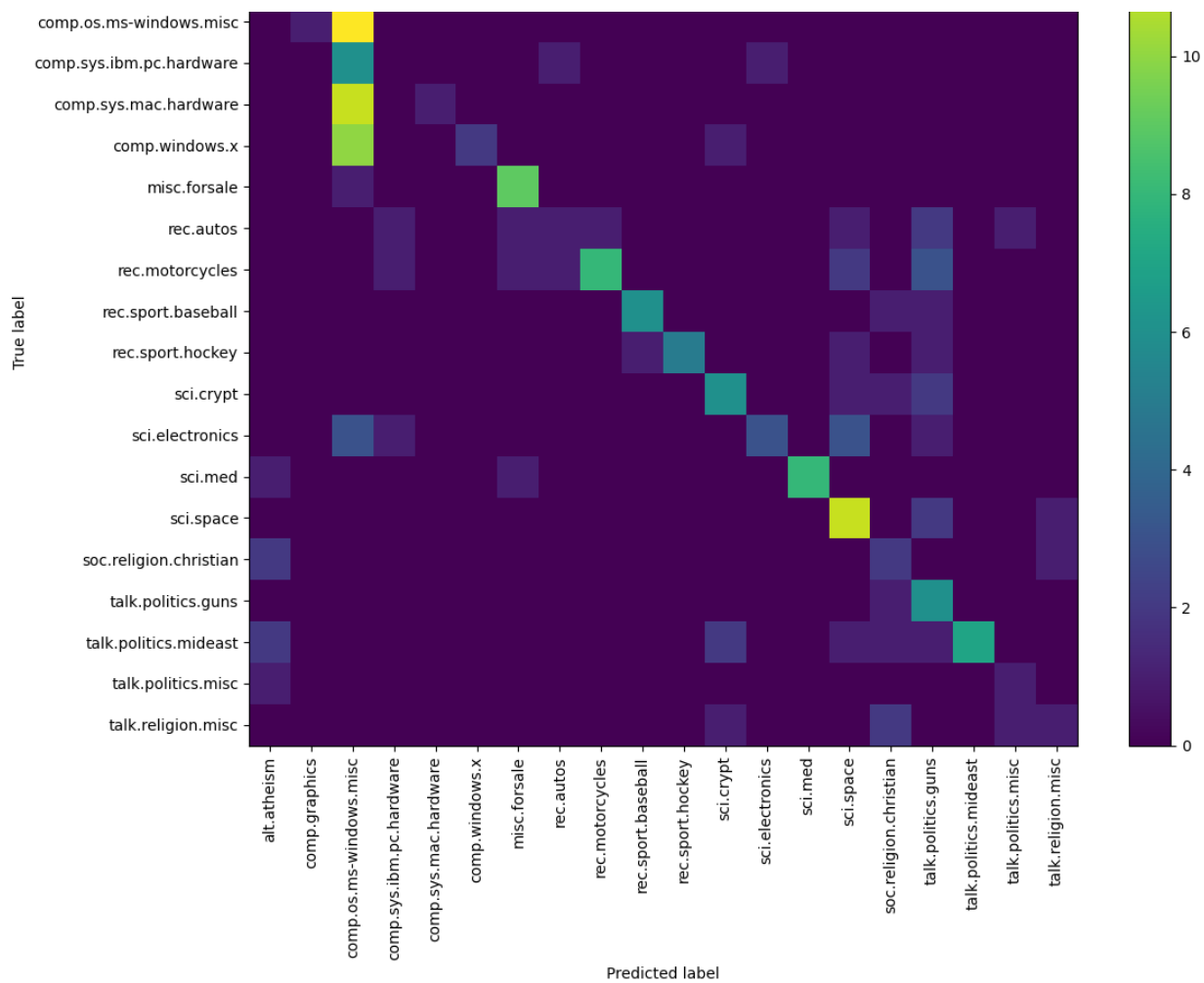
Now we'll use these functions to compute and visualize confusion matrices for each classifier.

```
confusion_matrices = dict()

for model in models:
    confusion_matrices[model] = part1_1_cm(classification_data, model)
    part_1_1_vis(confusion_matrices[model])
```







- ★ Make sure the images are rendered in the notebook pdf you submit.
- ★ For each of the different classifiers, which pair of classes is most often confused?
- ★ Use the scratch space below to explore the documents that are misclassified most often with these class pairings, and include snippets of a few examples below.
- ★ Why might these pairs of classes be more difficult for the classifiers to distinguish?

```
# TODO: Use this scratch space to find documents
# that are examples of common misclassifications
def most_confused_pair(cm):
    label_names = sorted(cm.keys())

    best_pair = None
    highest_count = -1

    for i in range(len(label_names)):
        for j in range(i + 1, len(label_names)):
            a = label_names[i]
            b = label_names[j]

            # Count confusion in both directions:
            # true A predicted B + true B predicted A
            count = cm[a][b] + cm[b][a]

            if count > highest_count:
```

```

        highest_count = count
        best_pair = (a, b)

    return best_pair, highest_count

for model in models:
    pair, count = most_confused_pair(confusion_matrices[model])
    print(model, ":", pair, "count =", count)
    class_a, class_b = pair
    pred_col = f"predictions_{model}"

    examples = classification_data[
        (
            (classification_data["newsgroup"] == class_a) &
            (classification_data[pred_col].apply(normalize_prediction) == class_b)
        )
        |
        (
            (classification_data["newsgroup"] == class_b) &
            (classification_data[pred_col].apply(normalize_prediction) == class_a)
        )
    ]

    print("\n=====")
    print("MODEL:", model)
    print("PAIR:", class_a, "<-->", class_b)

    for _, row in examples.head(3).iterrows():
        print("\nTRUE:", row["newsgroup"])
        print("PREDICTED:", normalize_prediction(row[pred_col]))
        print("TEXT:", row["text"][:400])

```

```
llama3.2:1b : ('comp.windows.x', 'talk.politics.misc') count = 12
```

```
=====
```

```
MODEL: llama3.2:1b
```

```
PAIR: comp.windows.x <--> talk.politics.misc
```

```
TRUE: comp.windows.x
```

```
PREDICTED: talk.politics.misc
```

```
TEXT: From: tim@kimba.catt.citri.edu.au (Tim Liddelaw)
```

```
Subject: Installing MIT X11R5 on Apollo
```

```
Organization: CATT Centre at CITRI, Melbourne, Australia
```

```
Lines: 14
```

```
Can any Apollo GURUS out there let me know of their experiences building MIT X11R5,
with or without GCC 2.3.3. In particular, is there anything I should watch out for.
```

```
Thanks in advance
```

```
--tim
```

```
TRUE: comp.windows.x
```

```
PREDICTED: talk.politics.misc
```

```
TEXT: From: dnh@mfltd.co.uk (Des Herriott)
```

```
Subject: XDM/xsession woes
```

```
Keywords: xdm, xterm
```

```
Lines: 22
```

```
Reply-To: dnh@mfltd.co.uk
```

```
Organization: Micro Focus Ltd.
```

```
X-Newsreader: mxrn 6.18-3
```

```
I've just managed to get xdm running from an NCR 3000 (an SVR4 486 box
```

```
running XFree86 1.2) to my NCD XDisplay. It's pretty much working, but
I'm encountering a weird error. I'm attempting to start an xterm from
my .xs
```

```
TRUE: comp.windows.x
PREDICTED: talk.politics.misc
TEXT: From: Bill.Kayser@delft.SGp.slb.COM (Bill Kayser)
Subject: Re: TeleUse, UIM/X, and C++
Article-I.D.: parsival.199304060629.AA00339
Organization: The Internet
Lines: 25
NNTP-Posting-Host: enterpoop.mit.edu
To: xpert@expo.lcs.mit.edu
Cc: Bill.Kayser@delft.sgp.slb.com
```

```
>
> Does anyone have any good ideas on how to integrate C++ code elegantly
> with TeleUse, UIM/X / Interface Architect generated co
llama3.2: ('rec.sport.baseball', 'talk.religion.misc') count = 11

=====
MODEL: llama3.2
PAIR: rec.sport.baseball <-> talk.religion.misc
```

Most commonly confused class pairs

- llama3.2:1b: comp.windows.x and talk.politics.misc — 12 total confusions.
- llama3.2: rec.sport.baseball and talk.religion.misc — 11 total confusions.
- gemma3:1b: comp.os.ms-windows.misc and rec.autos — 7 total confusions.
- gemma3:4b: comp.os.ms-windows.misc and comp.sys.mac.hardware — 11 total confusions.

For llama3.2:1b, several comp.windows.x posts about X11, XDM, xterm, and C++ GUI tools were incorrectly predicted as talk.politics.misc. These examples do not actually have much political content, suggesting that the small 1B model is not separating topics reliably and may be over-predicting a small number of classes.

For llama3.2, several baseball posts, including MLB scores, Nolan Ryan's injury, and discussion of the 1964 Phillies, were classified as talk.religion.misc. Again, there is little semantic overlap, so this confusion seems more like weak classification ability than genuine ambiguity between the categories.

For gemma3:1b, Windows-related posts discussing graphics benchmarks, Windows 3.1, DOS, and video-card drivers were predicted as rec.autos. These categories are not closely related, indicating that the 1B model is likely relying on weak lexical cues rather than accurately recognizing the overall topic.

The gemma3:4b confusion is more understandable. It often confuses comp.os.ms-windows.misc with comp.sys.mac.hardware. Both categories contain discussions of computers, displays, graphics hardware, drivers, power management, and peripherals. Because these categories share substantial technical vocabulary, distinguishing the operating-system-specific context can be difficult.

✓ Part 1.2: Accuracy (5 points)

Complete the following function, then use it to compute and compare the accuracies of each classifier.

Model predictions are raw text, not clean labels. Pass each one through `normalize_prediction` (defined in Part 1.0) before comparing it to the ground truth, and do it consistently across Parts 1.1 through 1.5 -- comparing the raw strings scores at least one of these models far below its real accuracy.

A prediction that is still not one of the labels after normalising counts as incorrect in Parts 1.2 through 1.5. Part

1.1 is the exception: a confusion matrix has no cell to put it in, so it is left out of the counts there.

- ★ Generate a table comparing the accuracy of each of the classifier systems.
- ★ Which one has the highest accuracy?

```
def part1_2_acc(x: DataFrame, y: str) -> float:
    prediction_col = f"predictions_{y}"
    correct = 0

    for _, row in x.iterrows():
        prediction = normalize_prediction(row[prediction_col])
        true_label = row["newsgroup"]

        if prediction == true_label:
            correct += 1

    return correct / len(x)

accuracies: dict[str, float] = dict()

for model in models:
    accuracies[model] = part1_2_acc(classification_data, model)

save_results(accuracies, "results/accuracies.json")

best_model = max(accuracies, key=accuracies.get)

accuracy_table = pd.DataFrame({
    "Model": list(accuracies.keys()),
    "Accuracy": list(accuracies.values())
}).sort_values("Accuracy", ascending=False).reset_index(drop=True)

print("Best model:", best_model)
print("Best accuracy:", accuracies[best_model])

accuracy_table
```

Best model: gemma3:4b
Best accuracy: 0.455

	Model	Accuracy	
0	gemma3:4b	0.455	
1	llama3.2	0.110	
2	gemma3:1b	0.060	
3	llama3.2:1b	0.030	

Next steps: [New interactive sheet](#)

`gemma3:4b` has the highest accuracy at 0.455 (45.5%). It performs substantially better than the other classifiers. `llama3.2` achieves 11.0% accuracy, `gemma3:1b` achieves 6.0%, and `llama3.2:1b` achieves 3.0%.

✓ Part 1.3: Precision (5 points)

Complete the following function that returns for each class the precision of the classifier's labels given

complete the remaining function that returns, for each class, the precision of the classifier's labels given ground-truth labels. (note: if there are no predictions for a given class, then precision should be 0)

- ★ Generate a table showing, for the best-performing classifier, the precision of its labels across each class.
- ★ For which class are its labels the most accurate? The least?

```
def part1_3_prec(x: DataFrame, y: str) -> dict[str, float]:
    prediction_col = f"predictions_{y}"
    precisions = {}

    for label in labels:
        true_positive = 0
        predicted_positive = 0

        for _, row in x.iterrows():
            prediction = normalize_prediction(row[prediction_col])
            true_label = row["newsgroup"]

            if prediction == label:
                predicted_positive += 1

            if true_label == label:
                true_positive += 1

        if predicted_positive == 0:
            precisions[label] = 0.0
        else:
            precisions[label] = true_positive / predicted_positive

    return precisions

if best_model not in models:
    print(f"best_model is still {best_model!r}. Set it in Part 1.2 to one of {models}, "
          f"then re-run this cell.")
    best_model_prec = None
else:
    best_model_prec = part1_3_prec(classification_data, best_model)

save_results(best_model_prec, "results/best_model_prec.json")

precision_table = pd.DataFrame({
    "Class": list(best_model_prec.keys()),
    "Precision": list(best_model_prec.values())
}).sort_values("Precision", ascending=False).reset_index(drop=True)

precision_table
```

	Class	Precision	
0	sci.med	1.000000	
1	comp.windows.x	1.000000	
2	comp.sys.mac.hardware	1.000000	
3	talk.politics.mideast	1.000000	
4	rec.sport.hockey	1.000000	
5	rec.motorcycles	0.888889	
6	rec.sport.baseball	0.857143	

6	rec.sport.baseball	0.857143
7	misc.forsale	0.750000
8	comp.graphics	0.666667
9	sci.electronics	0.600000
10	sci.space	0.550000
11	sci.crypt	0.545455
12	rec.autos	0.333333
13	talk.politics.guns	0.315789
14	talk.politics.misc	0.250000
15	comp.os.ms-windows.misc	0.240000
16	talk.religion.misc	0.200000
17	soc.religion.christian	0.181818
18	comp.sys.ibm.pc.hardware	0.000000
19	alt.atheism	0.000000

Next steps: [New interactive sheet](#)

For `gemma3:4b`, the highest precision is 1.0. Five classes achieve perfect precision: `sci.med`, `comp.windows.x`, `comp.sys.mac.hardware`, `talk.politics.mideast`, and `rec.sport.hockey`. This means that every time the model predicted one of these classes in this evaluation set, the prediction was correct.

The lowest precision is 0.0 for `comp.sys.ibm.pc.hardware` and `alt.atheism`. The model did not make any correct predictions among the examples it assigned to these classes. Overall, the precision values vary substantially across categories, showing that even though `gemma3:4b` is the best classifier by accuracy, its reliability still depends strongly on which class it predicts.

Part 1.4: Recall (5 points)

Complete the following function that returns, for each target class, the recall of the classifier's labels. (note: if there are no ground truth labels for a given class, the recall should be 0)

★ Generate a table showing, for each target class, the recall of the best-performing model's labels across each class.

★ For which class is recall the highest? The lowest?

```
def part1_4_rec(x: DataFrame, y: str) -> dict[str, float]:
    prediction_col = f"predictions_{y}"
    recalls = {}

    for label in labels:
        actual_count = 0
        correctly_recalled = 0

        for _, row in x.iterrows():
            true_label = row["newsgroup"]
            prediction = normalize_prediction(row[prediction_col])
```

```

        if true_label == label:
            actual_count += 1

        if prediction == label:
            correctly_recalled += 1

    if actual_count == 0:
        recalls[label] = 0.0
    else:
        recalls[label] = correctly_recalled / actual_count

    return recalls

if best_model not in models:
    print(
        f"best_model is still {best_model!r}. "
        f"Set it in Part 1.2 to one of {models}, then re-run this cell."
    )
    best_model_rec = None
else:
    best_model_rec = part1_4_rec(classification_data, best_model)

save_results(best_model_rec, "results/best_model_rec.json")

recall_table = pd.DataFrame({
    "Class": list(best_model_rec.keys()),
    "Recall": list(best_model_rec.values())
}).sort_values("Recall", ascending=False).reset_index(drop=True)

display(recall_table)

highest_recall = max(best_model_rec.values())
lowest_recall = min(best_model_rec.values())

highest_recall_classes = [
    label for label, value in best_model_rec.items()
    if value == highest_recall
]

lowest_recall_classes = [
    label for label, value in best_model_rec.items()
    if value == lowest_recall
]

print("Highest recall:", highest_recall)
print("Class(es):", highest_recall_classes)

print("Lowest recall:", lowest_recall)
print("Class(es):", lowest_recall_classes)

```

	Class	Recall	
0	comp.os.ms-windows.misc	0.923077	
1	misc.forsale	0.900000	
2	talk.politics.guns	0.857143	
3	sci.med	0.800000	
4	sci.space	0.785714	
5	sci.crypt	0.600000	
6	rec.sport.baseball	0.545455	

```

6         rec.sport.baseball 0.949495
7         rec.motorcycles 0.500000
8         talk.politics.mideast 0.500000
9         talk.politics.misc 0.500000
10        rec.sport.hockey 0.454545
11        soc.religion.christian 0.333333
12        sci.electronics 0.272727
13        comp.graphics 0.181818
14        talk.religion.misc 0.166667
15        comp.windows.x 0.153846
16        rec.autos 0.125000
17        comp.sys.mac.hardware 0.076923
18        comp.sys.ibm.pc.hardware 0.000000
19        alt.atheism 0.000000
Highest recall: 0.9230769230769231
Class(es): ['comp.os.ms-windows.misc']
Lowest recall: 0.0
Class(es): ['comp.sys.ibm.pc.hardware', 'alt.atheism']

```

Next steps: [New interactive sheet](#)

For `gemma3:4b`, the highest recall is for `comp.os.ms-windows.misc`, with a recall of about 0.923. This means the model correctly identifies about 92.3% of the examples that truly belong to that class.

The lowest recall is 0.0 for both `comp.sys.ibm.pc.hardware` and `alt.atheism`, meaning the model fails to correctly identify any examples from those two classes in this evaluation set.

The recall values vary a lot across classes. `gemma3:4b` performs especially well on `comp.os.ms-windows.misc`, `misc.forsale`, and `talk.politics.guns`, but performs poorly on several other categories, especially `comp.sys.ibm.pc.hardware`, `alt.atheism`, and `comp.sys.mac.hardware`.

✓ Part 1.5: F1 (5 points)

A weighted average F1 score computes an average F1 score over all target classes, weighted by the true number of instances of that class:

$$\overline{\text{F1}}(f) = \sum_{c \in \mathcal{C}} \frac{|\mathcal{D}_c|}{|\mathcal{D}|} \text{F1}(f, c, \mathcal{D})$$

Here, f is a classification function, $\mathcal{C}$ is the set of classes, $\mathcal{D}$ is the set of evaluation data with ground-truth labels, and $\mathcal{D}_c$ is the set of data whose ground-truth label is c .

Implement two functions below:

- A function that returns, for a target class, the F1 of a classifier's labels given ground truth labels
- A function that uses the previous function to compute a weighted average F1 score

```

def part1_5_class_f1(x: DataFrame, y: str, z: str) -> float:
    prediction_col = f"predictions_{y}"

    true_positive = 0
    false_positive = 0
    false_negative = 0

    for _, row in x.iterrows():
        true_label = row["newsgroup"]
        prediction = normalize_prediction(row[prediction_col])

        if prediction == z and true_label == z:
            true_positive += 1
        elif prediction == z and true_label != z:
            false_positive += 1
        elif prediction != z and true_label == z:
            false_negative += 1

    precision_denominator = true_positive + false_positive
    recall_denominator = true_positive + false_negative

    precision = (
        true_positive / precision_denominator
        if precision_denominator > 0
        else 0.0
    )

    recall = (
        true_positive / recall_denominator
        if recall_denominator > 0
        else 0.0
    )

    if precision + recall == 0:
        return 0.0

    return 2 * precision * recall / (precision + recall)

def part1_5_weighted_f1(x: DataFrame, y: str) -> float:
    total_examples = len(x)
    weighted_sum = 0.0

    for label in labels:
        class_count = (x["newsgroup"] == label).sum()
        class_f1 = part1_5_class_f1(x, y, label)

        weighted_sum += class_f1 * class_count

    return weighted_sum / total_examples

```

- ★ Generate a table comparing the weighted F1 score of each classifier.
- ★ Which is the best classifier according to weighted F1? How and why might this differ from an accuracy comparison (Part 1.2)?

```

f1s: dict[str, float] = dict()

for model in models:

```

```

f1s[model] = part1_5_weighted_f1(classification_data, model)

save_results(f1s, "results/f1s.json")

f1_table = pd.DataFrame({
    "Model": list(f1s.keys()),
    "Weighted F1": list(f1s.values())
}).sort_values("Weighted F1", ascending=False).reset_index(drop=True)

display(f1_table)

best_f1_model = max(f1s, key=f1s.get)

print("Best model by weighted F1:", best_f1_model)
print("Weighted F1:", f1s[best_f1_model])

```

	Model	Weighted F1	
0	gemma3:4b	0.455121	
1	llama3.2	0.097242	
2	gemma3:1b	0.051483	
3	llama3.2:1b	0.009506	

Best model by weighted F1: gemma3:4b
Weighted F1: 0.45512092874445803

Next steps: [New interactive sheet](#)

`gemma3:4b` is the best classifier according to weighted F1, with a score of approximately 0.4551. This matches the accuracy result from Part 1.2, where `gemma3:4b` also had the highest accuracy at 0.455.

Accuracy only measures the overall fraction of correct predictions. Weighted F1 also considers both precision and recall for each class, then weights each class by how many true examples it contains. Because of this, weighted F1 can differ from accuracy when a model performs unevenly across classes.

In my results, `gemma3:4b` has almost the same weighted F1 and accuracy, while the smaller models have noticeably lower weighted F1 than accuracy. For example, `llama3.2:1b` has 0.030 accuracy but only about 0.0095 weighted F1, which suggests that its few correct predictions are concentrated in limited classes rather than being balanced across the dataset.

✓ Part 2: Automatic speech recognition (25 points)

Here, we'll explore using a popular open-source automatic speech recognition system to compare its performance across English speakers whose native language differs.

✓ Part 2.0: Setup

`transcriptions.tsv` ships with the repo, so the cell below only downloads the Speech Accent Archive (<https://www.kaggle.com/datasets/rtatman/speech-accent-archive/data>) -- an 865MB archive that takes a while to extract -- when those transcripts are missing or incomplete.

```

import os

# transcriptions.tsv ships with the repo. The 865MB audio archive is only needed

```

```

# transcriptions.tsv ships with the repo. The 60MB audio archive is only needed
# when it is missing or short, so decide that first and skip the download otherwise.
transcript_counts = {}
if os.path.exists('transcriptions.tsv'):
    with open('transcriptions.tsv') as infile:
        for line in infile:
            language, _, _ = line.rstrip("\n").partition("\t")
            transcript_counts[language] = transcript_counts.get(language, 0) + 1

need_to_transcribe = not transcript_counts or any(n != 20 for n in transcript_counts.values)

if need_to_transcribe:
    import kagglehub
    speech_accent_path = kagglehub.dataset_download("rtatman/speech-accent-archive")
    speakers = pd.read_csv(f"{speech_accent_path}/speakers_all.csv")
    language_counts = speakers['native_language'].value_counts()
    languages_with_over_20_samples = set(language_counts[language_counts >= 20].index)
else:
    languages_with_over_20_samples = set(transcript_counts)

print(f"{len(languages_with_over_20_samples)} languages with at least 20 samples:")
print(sorted(languages_with_over_20_samples))

```

```

21 languages with at least 20 samples:
['amharic', 'arabic', 'cantonese', 'dutch', 'english', 'farsi', 'french', 'german', 'italian',

```

If the transcripts do need regenerating, load WhisperX. This might take some time to load.

```

if need_to_transcribe:
    import whisperx

    # Needs whisperx >= 3.8: 3.4.2 could not load its own VAD against current
    # pyannote-audio, and it also caps out at Python 3.12, which Colab is past.
    whisperx_model = whisperx.load_model("tiny", "cpu", compute_type="float32", language="en")

```

Now let's run inference. This is the smallest model we could run inference on, so its performance isn't going to be great. But, it shouldn't take *too* long to run on your local machine!

```

from tqdm import tqdm

transcriptions = {language: list() for language in languages_with_over_20_samples}

if need_to_transcribe:
    recordings_filepath = f"{speech_accent_path}/recordings/recordings/"
    for language in languages_with_over_20_samples:
        print(f"Transcribing for native language {language}...")
        for i in tqdm(range(20)):
            filename = f"{language}{i + 1}.mp3"
            audio = whisperx.load_audio(f"{recordings_filepath}/{filename}")
            transcription = whisperx_model.transcribe(audio, language="en")
            # flatten to text here, so this path and the tsv path agree on the type
            text = " ".join(segment["text"] for segment in transcription["segments"])
            transcriptions[language].append(" ".join(text.split()))

    with open("transcriptions.tsv", "w") as ofile:
        for language, transcripts in transcriptions.items():
            for transcript in transcripts:
                ofile.write(f"{language}\t{transcript}\n")
else:
    with open('transcriptions.tsv') as infile:
        for line in infile:

```

```
# partition rather than split so an empty transcript is still a valid row
language, _, transcript = line.rstrip("\n").partition("\t")
if language in transcriptions:
    transcriptions[language].append(transcript)
```

✓ Part 2.1: Word error rate (10 points)

Implement the following function to compute the word error rate of a candidate transcription against a reference. It takes the **reference** first and the candidate second, matching the call below. Each of the recordings we transcribed are readings of the same English text: *"Please call Stella. Ask her to bring these things with her from the store: Six spoons of fresh snow peas, five thick slabs of blue cheese, and maybe a snack for her brother Bob. We also need a small plastic snake and a big toy frog for the kids. She can scoop these things into three red bags, and we will go meet her Wednesday at the train station."*

Compare the two token lists case-insensitively. Whether a transcript writes *Please* or *please* is a choice the ASR system made about how to render the words it heard, not a recognition error, so folding case keeps the metric measuring what it is supposed to measure.

```
def part2_1(x: list[str], y: list[str]) -> float:
    reference = [token.lower() for token in x]
    candidate = [token.lower() for token in y]

    if len(reference) == 0:
        return 0.0 if len(candidate) == 0 else 1.0

    # dp[i][j] = minimum edits needed to transform
    # the first i reference tokens into the first j candidate tokens
    dp = [
        [0] * (len(candidate) + 1)
        for _ in range(len(reference) + 1)
    ]

    for i in range(len(reference) + 1):
        dp[i][0] = i

    for j in range(len(candidate) + 1):
        dp[0][j] = j

    for i in range(1, len(reference) + 1):
        for j in range(1, len(candidate) + 1):
            if reference[i - 1] == candidate[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                substitution = dp[i - 1][j - 1] + 1
                deletion = dp[i - 1][j] + 1
                insertion = dp[i][j - 1] + 1

                dp[i][j] = min(
                    substitution,
                    deletion,
                    insertion
                )

    edit_distance = dp[len(reference)][len(candidate)]

    return edit_distance / len(reference)
```

Then let's evaluate for all of the candidate transcriptions and each of the native languages.

- ★ Generate a table comparing the average WER across each of the languages.

```
import nltk
nltk.download('punkt_tab')

reference = "Please call Stella. Ask her to bring these things with her from the store: "
           "Six spoons of fresh snow peas, five thick slabs of blue cheese, and maybe a sr
           "for her brother Bob. We also need a small plastic snake and a big toy frog fo
           "She can scoop these things into three red bags, and we will go meet her Wednes
           "train station."

language_wers = {language: list() for language in languages_with_over_20_samples}
for language, transcripts in transcriptions.items():
    for candidate in transcripts:
        language_wers[language].append(part2_1(nltk.word_tokenize(reference), nltk.word_tok

save_results(language_wers, "results/language_wers.json")

average_wers = {
    language: sum(wers) / len(wers)
    for language, wers in language_wers.items()
}

wer_table = pd.DataFrame({
    "Native Language": list(average_wers.keys()),
    "Average WER": list(average_wers.values())
}).sort_values(
    "Average WER",
    ascending=True
).reset_index(drop=True)

display(wer_table)
```

[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!

	Native Language	Average WER	
0	german	0.083766	
1	english	0.085714	
2	macedonian	0.092857	
3	swedish	0.101299	
4	romanian	0.109091	
5	farsi	0.114286	
6	dutch	0.114286	
7	polish	0.131169	
8	japanese	0.133117	
9	french	0.140260	
10	arabic	0.141558	
11	italian	0.151948	
12	portuguese	0.160390	
13	russian	0.161039	
14	mandarin	0.166883	

15	korean	0.172727
16	cantonese	0.175325
17	turkish	0.186364
18	spanish	0.221429
19	amharic	0.227273
20	vietnamese	0.266234

Next steps: [New interactive sheet](#)

✓ Part 2.2: Confidence intervals (10 points)

Each average WER you just computed comes from 20 recordings. Twenty *different* speakers of the same native language would have given you a different average. Before ranking the languages, you need to know how much that number moves.

You cannot collect more recordings, but you can estimate the spread from the sample you already have. Draw 20 values *with replacement* from a language's 20 WERs and average them; that is one plausible alternative sample. Repeat it many times and the middle 95% of those averages is a 95% confidence interval for the mean.

Implement the function below. `x` is the list of WER values for one language and `y` is how many times to resample; return the lower and upper bounds of the 95% interval.

```
def part2_2_ci(x: list[float], y: int = 10000) -> tuple[float, float]:
    import numpy as np

    sample_size = len(x)
    bootstrap_means = []

    for _ in range(y):
        resampled_wers = np.random.choice(
            x,
            size=sample_size,
            replace=True
        )
        bootstrap_means.append(np.mean(resampled_wers))

    lower_bound = np.percentile(bootstrap_means, 2.5)
    upper_bound = np.percentile(bootstrap_means, 97.5)

    return float(lower_bound), float(upper_bound)

language_wer_cis = dict()

for language, wers in language_wers.items():
    language_wer_cis[language] = part2_2_ci(wers)

save_results(language_wer_cis, "results/language_wer_cis.json")

# Add confidence intervals to the Part 2.1 table
wer_ci_table = pd.DataFrame({
    "Native Language": list(average_wers.keys()),
    "Average WER": [average_wers[language] for language in average_wers],
    "95% CI Lower": [
        language_wer_cis[language][0]
    ],
    "95% CI Upper": [
        language_wer_cis[language][1]
    ]
})
```

```

        for language in average_wers
    ],
    "95% CI Upper": [
        language_wer_cis[language][1]
        for language in average_wers
    ]
}).sort_values(
    "Average WER",
    ascending=True
).reset_index(drop=True)

display(wer_ci_table)

```

	Native Language	Average WER	95% CI Lower	95% CI Upper	
0	german	0.083766	0.067532	0.101299	
1	english	0.085714	0.064286	0.112338	
2	macedonian	0.092857	0.072727	0.116234	
3	swedish	0.101299	0.075325	0.135731	
4	romanian	0.109091	0.088961	0.131818	
5	farsi	0.114286	0.086364	0.148701	
6	dutch	0.114286	0.091558	0.137662	
7	polish	0.131169	0.096753	0.168831	
8	japanese	0.133117	0.101948	0.166883	
9	french	0.140260	0.108442	0.173377	
10	arabic	0.141558	0.118182	0.168182	
11	italian	0.151948	0.118831	0.188312	
12	portuguese	0.160390	0.104545	0.237679	
13	russian	0.161039	0.120779	0.207143	
14	mandarin	0.166883	0.134416	0.200666	
15	korean	0.172727	0.130519	0.216883	
16	cantonese	0.175325	0.131153	0.225974	
17	turkish	0.186364	0.125974	0.265601	
18	spanish	0.221429	0.167532	0.284432	
19	amharic	0.227273	0.174026	0.283117	
20	vietnamese	0.266234	0.200000	0.340909	

Next steps: [New interactive sheet](#)

- ★ Extend your table from Part 2.1 with each language's confidence interval.
- ★ You named the lowest-WER languages in Part 2.1. Taking the intervals into account, which of those are actually distinguishable from one another, and which are not?
- ★ Some pairs of languages *are* clearly separated. What is different about those pairs, compared to the ones you cannot separate?
- ★ What would you need in order to separate the languages whose intervals currently overlap?

The lowest-WER languages from Part 2.1 are German, English, Macedonian, Swedish, and Romanian. After considering the 95% confidence intervals, these languages are not clearly distinguishable from one another because their intervals overlap substantially.

For example, German has an average WER of about 0.0838 with a 95% confidence interval of [0.0675, 0.1019], while English has an average WER of about 0.0857 with an interval of [0.0643, 0.1123]. These intervals overlap heavily, so the small difference in their average WERs is not strong evidence that one performs better than the other. The same is true for Macedonian, Swedish, and Romanian.

Some language pairs are much more clearly separated. For example, German's interval ends around 0.1019, while French begins around 0.1084, so those intervals do not overlap. Similarly, English and Arabic are separated because English's upper bound is about 0.1123 while Arabic's lower bound is about 0.1182. These pairs have differences in average WER that are large relative to the uncertainty in the estimates.

The pairs that cannot be separated tend to have very similar average WERs and relatively wide confidence intervals compared with the difference between their means. The clearly separated pairs have a larger gap between their average WERs, large enough that their confidence intervals no longer overlap.

To distinguish languages whose intervals currently overlap, I would need more independent speech samples from more speakers for each language. A larger sample size would reduce the uncertainty around the estimated mean WER and produce narrower confidence intervals, making smaller performance differences easier to detect.

Part 2.3: Error analysis (5 points)

- ★ Which languages have the lowest WER? Which have the highest?
- ★ Why might these languages have lower WER?
- ★ Which of the differences you just described survive the confidence intervals from Part 2.2, and which are within the noise?

Part 2.3 Error Analysis

The languages with the lowest average WER are German (0.0838), English (0.0857), Macedonian (0.0929), Swedish (0.1013), and Romanian (0.1091). The highest average WERs are Vietnamese (0.2662), Amharic (0.2273), Spanish (0.2214), and Turkish (0.1864).

One possible reason some languages have lower WER is that the English pronunciations of speakers in those groups may be closer to the pronunciation patterns represented in the ASR model's training data. The model may therefore recognize their speech more reliably. Differences in phonology, stress, rhythm, and pronunciation transfer from a speaker's native language could also affect recognition difficulty. However, these results should not be interpreted as showing that a language itself inherently produces easier or harder English speech, since each group contains only 20 speakers and other speaker-level differences may also contribute.

The confidence intervals show that several small differences among the lowest-WER languages are within the noise. For example, German has a 95% CI of approximately [0.0675, 0.1019] and English has [0.0643, 0.1123], so their intervals overlap substantially. German, English, Macedonian, Swedish, and Romanian therefore cannot be confidently ranked just from their mean WERs.

Larger differences are more convincing. For example, German's upper bound is about 0.1019 while French's lower bound is about 0.1084, so those intervals do not overlap. English and Arabic are also separated, with

English ending around 0.1123 and Arabic beginning around 0.1182. The difference between the strongest-performing groups and Vietnamese is also much larger than the uncertainty around their estimates.

At the high-WER end, however, the confidence intervals overlap considerably. Vietnamese, Amharic, Spanish, and Turkish therefore should not be treated as having a definite ranking even though their mean WERs differ. Overall, the confidence intervals suggest that large gaps in WER are meaningful, while many of the smaller differences between neighboring languages could simply reflect sampling variation.

✓ Part 3: Machine translation (25 points)

In this part we will evaluate models on their abilities to translate text via BLEU score.

✓ Part 3.0: Setup

Prepare the dataset we will use for evaluation. These are the Czech, Chinese and Gujarati halves of the standard WMT19 into-English translation sets. We will just use the first 25 examples of each for evaluation.

```
from datasets import load_dataset
from itertools import islice

# streaming=True reads only the validation shards. Without it, load_dataset also
# materialises every training split -- 7.3M cs-en rows, 26M zh-en rows, ~13GB of
# HuggingFace cache -- to reach the 75 sentences we actually evaluate on.
hf_dataset_cs = load_dataset("wmt/wmt19", "cs-en", split="validation", streaming=True)
hf_dataset_zh = load_dataset("wmt/wmt19", "zh-en", split="validation", streaming=True)
hf_dataset_gu = load_dataset("wmt/wmt19", "gu-en", split="validation", streaming=True)

dataset_items_cs = []
for i, item in enumerate(islice(hf_dataset_cs, 25)):
    dataset_items_cs.append({
        'id': i,
        'czech_text': item['translation']['cs'],
        'english_text': item['translation']['en']
    })

dataset_items_zh = []
for i, item in enumerate(islice(hf_dataset_zh, 25)):
    dataset_items_zh.append({
        'id': i,
        'chinese_text': item['translation']['zh'],
        'english_text': item['translation']['en']
    })

dataset_items_gu = []
for i, item in enumerate(islice(hf_dataset_gu, 25)):
    dataset_items_gu.append({
        'id': i,
        'gujarati_text': item['translation']['gu'],
        'english_text': item['translation']['en']
    })

dataset_cs = pd.DataFrame(dataset_items_cs)
dataset_zh = pd.DataFrame(dataset_items_zh)
dataset_gu = pd.DataFrame(dataset_items_gu)
print(f"Loaded translation data of shape {dataset_cs.shape} with columns {dataset_cs.columns}")
```

```
/usr/local/lib/python3.13/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
```

The secret 'HF_TOKEN' does not exist in your Colab secrets.
 To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>). You will be able to reuse this secret in all of your notebooks.
 Please note that authentication is recommended but still optional to access public models or datasets.
 warnings.warn(
 Loaded translation data of shape (25, 3) with columns ['id', 'czech_text', 'english_text'].

We will use a simple nltk based tokenizer to encode the n-gram tokens for computing the bleu score.

```
import nltk
from nltk.tokenize import word_tokenize

def tokenize_text(text):
    # Download required NLTK data if not already downloaded
    try:
        nltk.data.find('tokenizers/punkt_tab')
    except LookupError:
        nltk.download('punkt_tab')

    # Tokenize the text into a list of tokens
    tokens = word_tokenize(text)
    return tokens
```

```
tokenize_text("Welcome to CS-183: we are learning about NLP evaluations today!!!")
```

```
['Welcome',
 'to',
 'CS-183',
 ':',
 'we',
 'are',
 'learning',
 'about',
 'NLP',
 'evaluations',
 'today',
 '!!!',
 '!!!',
 '!!!']
```

```
dataset_cs.head()
```

	id	czech_text	english_text	
0	0	Hnutí za občanská práva vydalo cestovní výstra...	Civil rights group issues travel warning for M...	
1	1	Národní asociace pro zvýhodňování barevných li...	The National Association for the Advancement o...	
2	2	„Cestovní doporučení NAACP pro stát Missouri, ...	"The NAACP Travel Advisory for the state of Mi...	
3	3	NAACP uvedlo, že k tomuto kroku přistoupilo na...	A recent Missouri law making it harder for peo...	
4	4	„Dochází k porušování občanských práv.	"You have violations of civil rights that are ...	

Next steps: [New interactive sheet](#)

```
dataset_zh.head()
```

	id	chinese_text	english_text	
0	0	上周，古装剧《美人私房菜》临时停播，意外引发了关于国产剧收视率造假的热议讨论。	Last week, the broadcast of period drama "Beau...	

1	1	民权团体针对密苏里州发出旅行警告	Civil rights group issues travel warning for M...
2	2	由于密苏里州的歧视性政策和种族主义袭击，美国有色人种促进协会 (NAACP) 向准备前往密苏...	The National Association for the Advancement o...
"2017 年 8 月 28 日生效的 NAACP 密苏里州旅行咨询中呼吁，因近 "The NAACP Travel Advisory for the			
Next steps: New interactive sheet			

dataset_gu.head()			
	id	gujarati_text	english_text
0	0	બેરિયર ભાંગી ગયા પછી સેન્ટ્રલ પાર્કમાં હજારો ગ...	Thousands in Central Park panic after barrier ...
1	1	એક રાજકીય કાર્યક્રમમાં પોલીસ બેરિયરના ભંગાણને ...	The collapse of a police barrier sent thousand...
2	2	ડેમોક્રેટિક ન્યૂયોર્ક સરકારની મોટી નામી હસ્તીઓ...	Big-name personalities from Democratic New Yor...
3	3	શનિવારે સાંજે બેરિયર પડ્યા પછી અધિકારીઓએ ટોળાન...	Authorities quickly assured the crowd they wer...
4	4	સામાજિક ન્યાયના સળગતા મુદ્દાઓએ તડકાની ખરા બપોર...	Burning social justice issues dominated the ev...
Next steps: New interactive sheet			

now let's get the model predictions for these translations

<pre>def get_translation_prompt(text, source_language): translation_prompt = f"Translate the following text from {source_language} to English. translation_prompt += f"\n\nText:\n{text}" return translation_prompt prompts_cs = [get_translation_prompt(text, "Czech") for text in dataset_cs["czech_text"]] for model in models: print(f"Running inference for {model}") remaining_prompts = get_remaining_prompts(model, "pt3_1_translation_data_cs", prompts_cs) curr_inferences = await run_batch_inference(model, remaining_prompts, temperature=0.0, cache_inferences(model, "pt3_1_translation_data_cs", remaining_prompts, curr_inferences) inferences = load_inferences(model, "pt3_1_translation_data_cs", prompts_cs) dataset_cs[f"predictions_{model}"] = inferences prompts_zh = [get_translation_prompt(text, "Chinese") for text in dataset_zh["chinese_text"]] for model in models: print(f"Running inference for {model}") remaining_prompts = get_remaining_prompts(model, "pt3_1_translation_data_zh", prompts_zh) print(f"Remaining prompts: {len(remaining_prompts)}") curr_inferences = await run_batch_inference(model, remaining_prompts, temperature=0.0, cache_inferences(model, "pt3_1_translation_data_zh", remaining_prompts, curr_inferences) inferences = load_inferences(model, "pt3_1_translation_data_zh", prompts_zh) dataset_zh[f"predictions_{model}"] = inferences prompts_gu = [get_translation_prompt(text, "Gujarati") for text in dataset_gu["gujarati_text"]] for model in models: print(f"Running inference for {model}") remaining_prompts = get_remaining_prompts(model, "pt3_1_translation_data_gu", prompts_gu) curr_inferences = await run_batch_inference(model, remaining_prompts, temperature=0.0, cache_inferences(model, "pt3_1_translation_data_gu", remaining_prompts, curr_inferences) inferences = load_inferences(model, "pt3_1_translation_data_gu", prompts_gu) dataset_gu[f"predictions_{model}"] = inferences</pre>	
Running inference for llama3.2:1b	

```

100%|██████████| 25/25 [01:22<00:00, 3.30s/it]
Running inference for llama3.2
100%|██████████| 25/25 [01:02<00:00, 2.50s/it]
Running inference for gemma3:1b
100%|██████████| 25/25 [01:04<00:00, 2.59s/it]
Running inference for gemma3:4b
100%|██████████| 25/25 [00:58<00:00, 2.33s/it]
Running inference for llama3.2:1b
Remaining prompts: 25
100%|██████████| 25/25 [00:14<00:00, 1.71it/s]
Running inference for llama3.2
Remaining prompts: 25
100%|██████████| 25/25 [00:14<00:00, 1.70it/s]
Running inference for gemma3:1b
Remaining prompts: 25
100%|██████████| 25/25 [00:16<00:00, 1.55it/s]
Running inference for gemma3:4b
Remaining prompts: 25
100%|██████████| 25/25 [00:20<00:00, 1.25it/s]
Running inference for llama3.2:1b
100%|██████████| 25/25 [00:14<00:00, 1.78it/s]
Running inference for llama3.2
100%|██████████| 25/25 [00:11<00:00, 2.25it/s]
Running inference for gemma3:1b
100%|██████████| 25/25 [00:13<00:00, 1.92it/s]
Running inference for gemma3:4b
100%|██████████| 25/25 [00:13<00:00, 1.80it/s]

```

dataset_cs

	id	czech_text	english_text	predictions_llama3.2:1b	predictions_llama3.2	predictions_
0	0	Hnutí za občanská práva vydalo cestovní výstra...	Civil rights group issues travel warning for M...	Travel warning issued by the rights movement f...	Movement for Civil Rights issued a travel warn...	Committee for issued a
1	1	Národní asociace pro zvýhodňování barevných li...	The National Association for the Advancement o...	National Association for the Advancement of Co...	National Association for the Advancement of Co...	The National As the Adva
2	2	„Cestovní doporučení NAACP pro stát Missouri, ...	"The NAACP Travel Advisory for the state of Mi...	Travel advisories of the NAACP for the state o...	The NAACP travel advisory for the state of Mis...	"N recommend
3	3	NAACP uvedlo, že k tomuto kroku přistoupilo na...	A recent Missouri law making it harder for peo...	They have taken this action based on the curre...	The NAACP stated that it took this step based ...	The NAACP ha this step
4	4	„Dochází k porušování občanských práv.	"You have violations of civil rights that are ...	"Something is being destroyed."	There is a violation of civil rights.	"He is viola
5	5	Lidé jsou zastavováni policisty jen kvůli barv...	They're being pulled over because of their ski...	They are harassed by police just because of th...	People are stopped by police just because of t...	People are police only b
6	6	„Hromadí se nám dosud největší ...	"We are hearing complaints at a rate we	"Despite our complaints, we still have the mos...	The complaints are mounting to the greatest ...	We have so far largest

		stížno...	haven'...		ex...	
7	7	Jedná se o první varování svého druhu, které o...	It is the first such warning that the organiza...	It is the first warning of its species, which ...	It is the first warning of its kind that the o...	It is the first w spec
8	8	Mezi incidenty, které organizace citovala, byl...	The group cited incidents such as racial slurs...	racially motivated hate speech incidents cited...	Among the incidents cited by the organization,...	During incidents organ
9	9	Sanders zemřel za sporných okolností na začátk...	Sanders died under questionable circumstances ...	He died under suspicious circumstances at the ...	He died under disputed circumstances at the be...	Sander suspicious ci
10	10	Cestovní doporučení rovněž upozorňuje	The advisory also points to a recent report by...	Travel advice also warns about the latest stat...	The travel advisory also highlights a recent s...	Travel recomm warns of

Next steps: [New interactive sheet](#)

dataset_zh						
id	chinese_text	english_text	predictions_llama3.2:1b	predictions_llama3.2	prediction	
0	0	上周, 古装剧《美人私房菜》临时停播, 意外引发了关于国产剧收视率造假的热烈讨论。	Last week, the broadcast of period drama "Beau...	Last week, the period drama "Beautiful Private...	Last week, the costume drama "Beauty and the F...	Last week suspens
1	1	民权团体针对密苏里州发出旅行警告	Civil rights group issues travel warning for M...	Travel warning issued by civil rights group ag...	Human rights groups issue travel warning for M...	Civil rig issuing ti
2	2	由于密苏里州的歧视性政策和种族主义袭击, 美国有色人种促进会 (NAACP) 向准备前往密苏...	The National Association for the Advancement o...	Due to the state's discriminatory policies and...	Due to discriminatory policies and racist atta...	Due to policies a
3	3	"2017 年 8 月 28 日生效的 NAACP 密苏里州旅行咨询中呼吁, 因近期密苏里州发...	"The NAACP Travel Advisory for the state of Mi...	"NAACP密苏里州旅行咨询于2017年8月28日生效, 要求所有非裔美国人、游客以及密苏里州...	The NAACP Travel Advisory for Missouri, effect...	"On August NAACP ha
4	4	NAACP 指出, 最近通过的一项密苏里州法律使得人们更难赢得歧视诉讼, 该州执法也一定程度上针...	A recent Missouri law making it harder for peo...	The NAACP has stated that a recent Missouri la...	The NAACP stated that a recent law passed in M...	The NAACP I that a
5	5	侵犯公民权利的行为正发生在人们身上。	"You have violations of civil rihts that are ...	Violations of citizen rights are happening all...	The behavior that infringes on citizens' right...	Violations c are ha

6	6	他们因肤色被停车盘问，被殴打或被杀害，”密苏里州 NAACP 主席罗德·查培尔告诉堪萨斯城星...	They're being pulled over because of their ski...	They were stopped and beaten or killed by driv...	They were beaten or killed because of their sk...	“They w about the
7	7	“我们收到了许多投诉，数量前所未有。”	"We are hearing complaints at a rate we haven't...	"We have received many complaints, the number ...	"We have received many complaints, a number un...	We have complaints,
8	8	这是该组织在美国针对某个州发布的第一个此类警告。	It is the first such warning that the organiza...	This is the first warning issued by the organi...	This is the first of its kind warning issued b...	This is the first is
9	9	该组织援引了密苏里大学对黑人学生的种族诽谤以及 28 岁田纳西州黑人男性托利·桑德斯的死亡事件。	The group cited incidents such as racial slurs...	The organization cited the racist remarks made...	The organization cited the racial slurs direct...	The organ the assi
10	10	今年早些时候，桑德斯在可疑情况下死亡。他在密苏里州旅行时燃油耗尽，被密苏里州警方在无指控犯罪...	Sanders died under questionable circumstances ...	He died under suspicious circumstances in Miss...	This is the translation: \n\nIn the early part ...	In recent has been

Next steps: [New interactive sheet](#)

dataset_gu						
	id	gujarati_text	english_text	predictions_llama3.2:1b	predictions_llama3.2	prediction
0	0	બેરિયર ભાંગી ગયા પછી સેન્ટ્રલ પાર્કમાં હજારો ગ...	Thousands in Central Park panic after barrier ...	After beating his brother, he went to the cent...	After the barrier was broken, thousands were i...	The potato thous
1	1	એક રાજકીય કાર્યક્રમમાં પોલીસ બેરિયરના ભંગાણને ...	The collapse of a police barrier sent thousand...	Thousands of people were trapped in fear of be...	Thousands of people were forced to take shelte...	“A threatened
2	2	ડેમોક્રેટિક ન્યૂયોર્ક સરકારની મોટી નામી હસ્તીઓ...	Big-name personalities from Democratic New Yor...	The Democratic New York State Legislature has ...	Democratic New York government's prominent fig...	Dem governmer
3	3	શનિવારે સાંજે બેરિયર પડ્યા પછી અધિકારીઓએ ટોળાન...	Authorities quickly assured the crowd they wer...	Gujarati to English translation: \nAs the day e...	After the Beriyar (a type of fruit) fell on Sa...	On Saturd encou
4	4	સામાજિક ન્યાયના સળગતા મુદ્દાઓએ તડકાની ખરાબ પોર...	Burning social justice issues dominated the ev...	Social justice movements have started to take ...	The phenomenon of social justice issues escala...	Social ju bro
5	5	લગભગ 60,000 લોકોથી ગ્લોબલ સિટીઝન ફેસ્ટિવલ	About 60,000 people filled the park's Great	The main gate of the global city festival has ...	The park became almost 60,000 people for the G...	The p crowded v

		માટ...	La...			
6	6	કેટલાક સ્પીકરોએ પ્રેક્ષકોને સુપ્રીમ કોર્ટના નો...	Some speakers asked the audience to call their...	Some members of the opposition have expressed ...	Some speakers called upon their Congress party...	Some politi cong
7	7	બીજી ઓક્ટોબરથી અત્રા હજારે કરશે ભૂખ હડતાળ, PM ...	Anna Hazare will go on hunger strike from 2nd ...	He will write a letter to PM Modi from Annha.	From October, Anna Hazare will start hunger st...	Second Oc mid
8	8	મહારાષ્ટ્ર: સામાજિક કાર્યકર્તા અત્રા હજારેએ બી...	Maharashtra: The social activist Anna Hazare h...	The social worker Annapoorna Hazare has filed ...	Maharashtra: Social activist Anna Hazare has a...	Mahara Workers A
9	9	અત્રા હજારેએ વડાપ્રધાન પર પ્રહાર કરતા કહ્યું ક...	Anna Hazare has heavily criticized the Prime m...	Annaji has said that the government led by the...	Anna Hazare has said that the BJP-led governme...	The Pr going to
10	10	પરંતુ ચાર વર્ષ વીતી ગયા પણ સરકાર કોઈ ના કોઈ કા...	Four years have passed but the government is a...	However, four years have passed since the peop...	But after four years, the government has not s...	But d passing, t
11	11	તેમણે કહ્યું કે તે લોકપાલ નિયુક્તને વહને બીજી ...	He added that regarding the Lokpal election, h...	He said that he would leave the local panchaya...	He said that from the second October, he will ...	They said th g
12	12	અત્રા હજારેએ વડાપ્રધાન મોદીને ગુરુવારે પત્ર લખ...	Anna Hazare wrote a letter to PM Modi on Thurs...	The Prime Minister of Annaji has written to th...	Anna Hazare has written a letter to Prime Mini...	Prime Minis lette

Next steps: [New interactive sheet](#)

✓ Part 3.1 n-gram precision (5 points)

complete the following function to calculate the n-gram precision of a predicted translation given the reference.

`pred` and `ref` are the raw translation strings, the same ones `bleu` is given in Part 3.3. Use the `tokenize_text` helper from Part 3.0 on them.

```
def ng_prec(pred, ref, n):
    pred_tokens = tokenize_text(pred)
    ref_tokens = tokenize_text(ref)

    if len(pred_tokens) < n:
        return 0.0

    pred_ngrams = [
        tuple(pred_tokens[i:i + n])
        for i in range(len(pred_tokens) - n + 1)
    ]

    ref_ngrams = [
        tuple(ref_tokens[i:i + n])
        for i in range(len(ref_tokens) - n + 1)
    ]
```

```

ref_ngram_counts = {}

for ngram in ref_ngrams:
    ref_ngram_counts[ngram] = ref_ngram_counts.get(ngram, 0) + 1

matches = 0

for ngram in pred_ngrams:
    if ref_ngram_counts.get(ngram, 0) > 0:
        matches += 1
        ref_ngram_counts[ngram] -= 1

return matches / len(pred_ngrams)

```

```

ngram_precisions_cs = dict()
for model in models:
    ngram_precisions_cs[model] = [
        ng_prec(prediction, reference, n) for n in (1, 2, 3, 4)]
    for prediction, reference in zip(dataset_cs[f"predictions_{model}"], dataset_cs["er
    ])

save_results(ngram_precisions_cs, "results/ngram_precisions_cs.json")

```

✓ Part 3.2 brevity penalty (5 points)

complete the following function to calculate the brevity penalty for the bleu score

`pred` and `ref` are the raw strings here too.

```

def brev_pen(pred, ref):
    pred_tokens = tokenize_text(pred)
    ref_tokens = tokenize_text(ref)

    pred_len = len(pred_tokens)
    ref_len = len(ref_tokens)

    if pred_len == 0:
        return 0.0

    if pred_len > ref_len:
        return 1.0

    return math.exp(1 - (ref_len / pred_len))

```

```

import math
brevity_penalties_cs = dict()
for model in models:
    brevity_penalties_cs[model] = [
        brev_pen(prediction, reference)
        for prediction, reference in zip(dataset_cs[f"predictions_{model}"], dataset_cs["er
    ])

save_results(brevity_penalties_cs, "results/brevity_penalties_cs.json")

```

✓ Part 3.3 putting it together (10 points)

complete the following function to calculate the full bleu score, using the two functions from the previous

parts. Represent the bleu score as discussed in class using the 1, 2, 3, and 4-gram precisions.

parts. Represent the bleu score as discussed in class, using the 1, 2, 3, and 4 gram precisions.

Sentence-level BLEU over 25 short sentences hits zero 4-gram precision often — for some model and language pairs, on most of the sentences. Taken literally the geometric mean then makes the whole score exactly 0, and half your table ties at 0.0 with no way to tell the bad translations from the hopeless ones. Substitute a small constant (use `1e-7`) for any n-gram precision that comes out 0, so the scores stay comparable.

```
def bleu(pred, ref):
    import math

    precisions = []

    for n in (1, 2, 3, 4):
        p_n = ng_prec(pred, ref, n)

        if p_n == 0:
            p_n = 1e-7

        precisions.append(p_n)

    log_precision_average = sum(
        math.log(p) for p in precisions
    ) / 4

    geometric_mean = math.exp(log_precision_average)

    return brev_pen(pred, ref) * geometric_mean
```

★ Generate a table computing the bleu score for all model outputs for each language pair.

```
bleu_scores_cs = {model: [] for model in models}
bleu_scores_zh = {model: [] for model in models}
bleu_scores_gu = {model: [] for model in models}

references_cs = dataset_cs['english_text']
references_zh = dataset_zh['english_text']
references_gu = dataset_gu['english_text']

for model in models:
    predictions_cs = dataset_cs[f'predictions_{model}']
    for reference, prediction in zip(references_cs, predictions_cs):
        bleu_scores_cs[model].append(bleu(prediction, reference))

    predictions_zh = dataset_zh[f'predictions_{model}']
    for reference, prediction in zip(references_zh, predictions_zh):
        bleu_scores_zh[model].append(bleu(prediction, reference))

    predictions_gu = dataset_gu[f'predictions_{model}']
    for reference, prediction in zip(references_gu, predictions_gu):
        bleu_scores_gu[model].append(bleu(prediction, reference))

save_results(bleu_scores_cs, "results/bleu_scores_cs.json")
save_results(bleu_scores_zh, "results/bleu_scores_zh.json")
save_results(bleu_scores_gu, "results/bleu_scores_gu.json")

bleu_summary = []

for model in models:
    bleu_summary.append({
```

```

        "Model": model,
        "Czech → English": sum(bleu_scores_cs[model]) / len(bleu_scores_cs[model]),
        "Chinese → English": sum(bleu_scores_zh[model]) / len(bleu_scores_zh[model]),
        "Gujarati → English": sum(bleu_scores_gu[model]) / len(bleu_scores_gu[model])
    })

bleu_table = pd.DataFrame(bleu_summary)

bleu_table["Overall Average"] = bleu_table[
    ["Czech → English", "Chinese → English", "Gujarati → English"]
].mean(axis=1)

bleu_table = bleu_table.sort_values(
    "Overall Average",
    ascending=False
).reset_index(drop=True)

display(bleu_table)

# The lowest-scoring translations, which Part 3.4 asks you to look at.
SOURCE_COLUMN = {"cs": "czech_text", "zh": "chinese_text", "gu": "gujarati_text"}
for language, dataset, scores in [("cs", dataset_cs, bleu_scores_cs),
                                   ("zh", dataset_zh, bleu_scores_zh),
                                   ("gu", dataset_gu, bleu_scores_gu)]:
    ranked = sorted((score, model, i) for model in models
                    for i, score in enumerate(scores[model]) if score is not None)
    if not ranked:
        continue
    print(f"\n===== {language}: five lowest-scoring translations =====")
    for score, model, i in ranked[:5]:
        print(f"\n BLEU {score:.4f} {model}")
        print(f" source: {dataset[SOURCE_COLUMN[language]][i][:120]}")
        print(f" reference: {dataset['english_text'][i][:120]}")
        print(f" prediction: {dataset[f'predictions_{model}'][i][:120]}")

```

	Model	Czech → English	Chinese → English	Gujarati → English	Overall Average	
0	gemma3:4b	0.222620	0.199080	0.174491	0.198730	
1	llama3.2	0.209533	0.190589	0.159377	0.186500	
2	gemma3:1b	0.151284	0.146198	0.085979	0.127821	
3	llama3.2:1b	0.128952	0.093658	0.046352	0.089654	

===== cs: five lowest-scoring translations =====

BLEU 0.0000 llama3.2:1b
source: Zavražděnou je modelka Sally Anne Bowman.
reference: The victim is model Sally Anne Bowman.
prediction: She was assassinated.

BLEU 0.0000 gemma3:4b
source: „Dochází k porušování občanských práv.
reference: "You have violations of civil rights that are happening to people.
prediction: Human rights are being violated.

BLEU 0.0000 llama3.2:1b
source: „Dochází k porušování občanských práv.
reference: "You have violations of civil rights that are happening to people.
prediction: "Something is being destroyed."

BLEU 0.0000 gemma3:1b
source: Ten u soudu přiznal pouze napadení mladistvé a právník tvrdil, že jeho klier
reference: He only confessed to assaulting the young woman to the court. and his lawyer

prediction: Ten u soudu přiznal pouze napadení mladistvé a právník tvrdil, že jeho klier

BLEU 0.0000 gemma3:1b
 source: „Dochází k porušování občanských práv.
 reference: "You have violations of civil rights that are happening to people.
 prediction: "He is violating citizens' rights."

===== zh: five lowest-scoring translations =====

BLEU 0.0000 gemma3:1b
 source: 参考消息网1月18日报道英媒称，北京或许有三千年的悠久历史，但它也在迅速变身为今天的现代魅力
 reference: Reference News Network reported on January 18 that the British media organiz
 prediction: 参考消息网1月18日报道，北京或许有三千年的悠久历史，但它也在迅速变身为今天的现代魅力都市。

BLEU 0.0000 llama3.2:1b
 source: “2017 年 8 月 28 日生效的 NAACP 密苏里州旅行咨询中呼吁，因近期密苏里州发生了一系列可疑
 reference: "The NAACP Travel Advisory for the state of Missouri, effective through Augu
 prediction: "NAACP密苏里州旅行咨询于2017年8月28日生效，要求所有非裔美国人、游客以及密苏里州人在密苏

BLEU 0.0000 gemma3:1b
 source: “我们收到了许多投诉，数量前所未有。”
 reference: "We are hearing complaints at a rate we haven't heard before."
 prediction: We have received many complaints, the number is unprecedented.

BLEU 0.0000 gemma3:4b
 source: 侵犯公民权利的行为正发生在人们身上。
 reference: "You have violations of civil rights that are happening to people.
 prediction: Actions violating citizens' rights are currently taking place.

BLEU 0.0000 llama3.2:1b
 source: 咨询中还指出，密苏里州总检察长办公室最近的一份报告显示，“与白人相比，该州的黑人司机被停车
 reference: The advisory also points to a recent report by the Missouri Attorney General
 prediction: Black drivers in Missouri are stopped more often than white drivers during t

===== gu: five lowest-scoring translations =====

BLEU 0.0000 llama3.2:1b
 source: બેરિયર ભાંગી ગયા પછી સેન્ટ્રલ પાર્કમાં હજારો ગભરાટમાં
 reference: Thousands in Central Park panic after barrier collapse
 prediction: After beating his brother, he went to the central park and saw a huge crowd.

BLEU 0.0000 gemma3:1b
 source: જેને લઈને ખેડૂતો આત્મહત્યા કરી રહ્યા છે.
 reference: Because of which they are killing themselves.
 prediction: Farmers are committing suicide.

BLEU 0.0000 gemma3:1b
 source: બેરિયર ભાંગી ગયા પછી સેન્ટ્રલ પાર્કમાં હજારો ગભરાટમાં
 reference: Thousands in Central Park panic after barrier collapse
 prediction: The potatoes collapsed and thousands of them were panicked in the central pa

BLEU 0.0000 llama3.2:1b
 source: જેને લઈને ખેડૂતો આત્મહત્યા કરી રહ્યા છે.
 reference: Because of which they are killing themselves.
 prediction: Those who have gone to the field have committed suicide.

BLEU 0.0000 llama3.2
 source: ગાંભોઈ દુષ્કર્મ કેસ બાદ બાળ અાયોગ અાવ્યું હરકતમાં ,અત્યાર સુધીની મહત્વની ખબરો જોવા અહીં ક્લિક
 reference: After Gambhoi rape case, the truth has come out. Click here to watch the imp
 prediction: No translation available. The provided text is in Gujarati and does not cont

Next steps: [New interactive sheet](#)

Part 2.4 Error Analysis (5 points)

Part 3.4 Error Analysis (5 points)

The cell above prints the five lowest-scoring translations for each language pair.

★ Look at those and at other low-scoring documents. Identify 3 types of errors this MT system seems to be making and show two example documents per error.

✓ Part 3.4 Error Analysis

I found three recurring types of translation errors in the low-BLEU outputs.

1. Failure to translate the source language

Some model outputs simply reproduce the source text instead of translating it into English.

- For the Czech sentence beginning with "Ten u soudu přiznal pouze napadení mladistvé...", `gemma3:1b` returned the original Czech sentence unchanged instead of an English translation.
- For the Chinese sentence beginning with "参考消息网1月18日报道...", `gemma3:1b` also returned the original Chinese text unchanged.

These examples suggest that the model sometimes fails to follow the translation instruction at all, especially for the smaller model.

2. Hallucination or incorrect meaning

In several cases, the model produced fluent English that did not preserve the meaning of the source.

- For the Gujarati sentence meaning "Thousands in Central Park panic after barrier collapse," `llama3.2:1b` produced: "After beating his brother, he went to the central park and saw a huge crowd." The ideas about beating a brother and going to the park are not present in the source.
- For the same Gujarati sentence, `gemma3:1b` produced: "The potatoes collapsed and thousands of them were panicked in the central park." This introduces "potatoes," which is unrelated to the original meaning.

A similar problem appears in Czech, where `llama3.2:1b` translated a sentence about violations of civil rights as "Something is being destroyed." These outputs preserve little of the original semantics.

3. Loss of important information or over-simplification

Some translations preserve the general topic but omit important details from the source.

- For "Zavražděnou je modelka Sally Anne Bowman," whose reference is "The victim is model Sally Anne Bowman," `llama3.2:1b` translated it as "She was assassinated." This loses the victim's name and the fact that she was a model.
- For the Chinese sentence about a Missouri Attorney General report saying black drivers were 75% more likely to be stopped than white drivers, `llama3.2:1b` produced only "Black drivers in Missouri are stopped more often than white drivers during traffic stops." The basic idea is retained, but the source of the statistic and the specific 75% figure are lost.

Overall, the weaker models show more severe errors such as copying the source language, hallucinating unrelated content, and dropping important details.

Double-click (or enter) to edit

✓ Part 4: Open-ended text generation (25 points)

In this part we will evaluate open ended text using an LLM judge

in this part we will evaluate open ended text using an LLM judge.

```
# The judge model, 5.2GB. Only needed from here on.
import subprocess, sys
if "google.colab" in sys.modules:
    subprocess.run(["ollama", "pull", "qwen3:8b"], check=True)
else:
    print("Run `ollama pull qwen3:8b` in a terminal if you have not already.")
```

```
import subprocess
import time

# Restart the Ollama server after the Colab session restart
subprocess.Popen(
    ["ollama", "serve"],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL
)

time.sleep(5)

# Check which models are currently available
subprocess.run(["ollama", "list"])
```

```
CompletedProcess(args=['ollama', 'list'], returncode=0)
```

✓ Part 4.0: Setup

Here we list the prompts we will use for comparing LLMs. Since the judge should ideally be more powerful than the models generating the outputs, we will use Qwen3 8B as the judge -- twice the size of the largest model it is judging, and small enough to run wherever the rest of the assignment does.

```
prompts = [
    "Explain bleu score in 100 words or less.",
    "Why does bleu score use a brevity penalty?",
    "How is precision different from recall?",
    "Why would one want to use F1-score instead of accuracy?",
    "What are the best boba spots in Berkeley?",
    "Why would an LLM judge prefer one output over another?",
    "What does CS 61C at UC Berkeley teach?",
    "What do most students at UC Berkeley think of EE16a?",
    "What is the best place to get ramen in Berkeley?",
    "Write a poem about Berkeley.",
]

data = pd.DataFrame(prompts, columns=["prompt"])

judge_model = "qwen3:8b"
```

Get the outputs from each of the models for these prompts.

```
outputs_per_model = {}
for model in models:
    print(f"Running inference for {model}")
    remaining_prompts = get_remaining_prompts(model, "pt3_1_pairwise_judgement_data", prompt)
    curr_inferences = await run_batch_inference(model, remaining_prompts, temperature=0.0,
    cache_inferences(model, "pt3_1_pairwise_judgement_data", remaining_prompts, curr_inferenc
    inferences = load_inferences(model, "pt3_1_pairwise_judgement_data", prompts)
```

```
outputs_per_model[model] = inferences
```

```
Running inference for llama3.2:1b
```

```
0it [00:00, ?it/s]
```

```
Running inference for llama3.2
```

```
0it [00:00, ?it/s]
```

```
Running inference for gemma3:1b
```

```
0it [00:00, ?it/s]
```

```
Running inference for gemma3:4b
```

```
0it [00:00, ?it/s]
```

Have a look at some of the outputs

```
outputs_per_model
```

```
{'llama3.2:1b': ["The Bleu score is a measure of the similarity between two text documents. It assigns a score from 0 to 1, where 1 indicates the documents are identical and 0 indicates they are completely different. The score is calculated by comparing the frequency of each word in the two documents. A higher score indicates more similar words, resulting in a higher Bleu score. This metric is commonly used in natural language processing and machine translation to evaluate the similarity between text documents. It's often used as a baseline for evaluating the quality of machine translation outputs."],
```

```
"Bleu score, a metric used to evaluate the quality of a sentence, uses a brevity penalty to discourage the use of unnecessary words. This penalty is applied to the score calculation.\n\nThe brevity penalty is based on the idea that longer sentences are less informative and less useful for the model. By penalizing sentences with more words, the model is encouraged to use more concise language to convey the same information.\n\nIn other words, the brevity penalty helps the model to:\n\n1. Focus on the most important information: By penalizing longer sentences, the model is forced to prioritize the most critical words and phrases, ensuring that the most important information is conveyed.\n2. Reduce noise and irrelevant information: Longer sentences can contain unnecessary words, phrases, or words that don't add significant value to the sentence. The brevity penalty helps to eliminate this noise and focus on the most relevant information.\n3. Improve the model's ability to generalize: By using concise language, the model is better able to generalize to new, unseen data. This is because concise sentences are more likely to be applicable to a wider range of situations.\n\nThe brevity penalty is typically applied as a fraction of the sentence length, with a higher penalty indicating a longer sentence. For example, if a sentence has 50 words, the brevity penalty might be 10-20%. This means that the model will penalize sentences with 50 words or more by reducing their score.\n\nOverall, the brevity penalty is a useful tool for improving the quality and usefulness of the bleu score, and it helps the model to focus on the most important information and reduce noise in the data."],
```

```
"In the context of data analysis and information retrieval, precision and recall are two important metrics that are often used to evaluate the performance of a system or algorithm.\n\n**Precision** refers to the proportion of true positives (correctly identified instances) among all positive predictions made by a system. In other words, it measures how accurate a system is at identifying instances that actually exist in the data.\n\n**Recall**, on the other hand, refers to the proportion of true positives among all actual instances in the data. It measures how well a system is at identifying all instances that actually exist in the data.\n\nTo illustrate the difference, consider a simple example:\n\nSuppose you're searching for a specific book on a library's catalog. A search engine might return a list of all books that match the search query, including some that don't actually exist. In this case:\n\n* Precision would be the proportion of books that actually exist in the catalog (e.g., 80%) because the search engine is accurate in identifying the correct books.\n* Recall would be the proportion of books that actually exist in the catalog (e.g., 100%) because the search engine is also accurate in identifying all books that exist in the catalog.\n\nIn general, precision is more important than recall because it measures the system's ability to identify true positives, while recall measures the system's ability to identify all actual instances. However, in some cases, recall might be more important than precision, especially if the system is designed to identify all instances that exist in the data, even if they're not all correct.\n\nIt's worth noting that precision and recall are not always independent of each other. In some cases, a system
```

might have high precision but low recall, because it's not identifying all instances that exist in the data. Conversely, a system might have low precision but high recall, because it's not identifying all instances that actually exist in the data.",

"In the context of machine learning and predictive modeling, F1-score (also known as F-measure) is often preferred over accuracy instead of accuracy alone for several reasons:

- 1. **Balances precision and recall**: F1-score takes into account both precision (the proportion of true positives among all predicted positive instances) and recall (the proportion of true positives among all actual positive instances). This balance ensures that the model is not overly sensitive to false positives or false negatives.
- 2. **Reflects nuanced relationships**: F1-score captures the complexity of the relationship between the model's predictions and the actual labels. It's a more nuanced measure than

✓ Part 4.1 Pairwise judge (10 Points)

Let's implement a function that takes as input a prompt and two candidate generated texts and calls an LLM to judge which of the two texts is preferred.

We've implemented the prompt for the judge below. You should compute the head to head comparisons for all models across all prompt pairs. Include the pairwise judgement for each pair/prompt in the `head_to_head_judgements` object below. Store the name of the winning model as each value, not the judge's raw 1 or 2 -- that number is relative to the order you passed the two outputs in.

```
def get_pairwise_judgement_prompt(prompt, output1, output2):
    judgement_prompt = f"Which of the following two outputs is a better response to the prompt: {prompt}?"
    return judgement_prompt

head_to_head_judgements = {}

judge_prompts = []
judge_locations = []

for prompt_index, original_prompt in enumerate(prompts):
    for model_a in models:
        for model_b in models:
            if model_a == model_b:
                continue

            output_a = outputs_per_model[model_a][prompt_index]
            output_b = outputs_per_model[model_b][prompt_index]

            judge_prompts.append(
                get_pairwise_judgement_prompt(
                    original_prompt,
                    output_a,
                    output_b
                )
            )

            judge_locations.append(
                (prompt_index, model_a, model_b)
            )

judge_answers = await run_batch_inference(
    judge_model,
    judge_prompts,
    temperature=0.0,
    max_tokens=8
)

for answer, location in zip(judge_answers, judge_locations):
    prompt_index, model_a, model_b = location
```

```

cleaned_answer = (
    answer.strip()
    if answer is not None
    else ""
)

if cleaned_answer == "1":
    winner = model_a
elif cleaned_answer == "2":
    winner = model_b
else:
    winner = None

head_to_head_judgements[prompt_index][
    str((model_a, model_b))
] = winner

save_results(head_to_head_judgements, "results/head_to_head_judgements.json")

```

100%  120/120 [02:29<00:00, 1.24s/it]

1%		1/120	[00:24<48:22, 24.39s/it]
2%		2/120	[00:24<20:07, 10.23s/it]
2%		3/120	[00:24<11:05, 5.69s/it]
3%		4/120	[00:25<06:52, 3.56s/it]
4%		5/120	[00:25<04:34, 2.39s/it]
5%		6/120	[00:25<03:08, 1.65s/it]
6%		7/120	[00:26<02:14, 1.19s/it]
7%		8/120	[00:26<01:39, 1.13it/s]
8%		9/120	[00:26<01:15, 1.47it/s]
8%		10/120	[00:26<01:02, 1.77it/s]
9%		11/120	[00:27<00:48, 2.23it/s]
10%		12/120	[00:27<00:39, 2.77it/s]
11%		13/120	[00:27<00:50, 2.11it/s]
12%		14/120	[00:28<00:59, 1.77it/s]
12%		15/120	[00:29<01:10, 1.49it/s]
13%		16/120	[00:30<01:14, 1.40it/s]
14%		17/120	[00:31<01:18, 1.31it/s]
15%		18/120	[00:32<01:22, 1.24it/s]
16%		19/120	[00:33<01:34, 1.07it/s]
17%		20/120	[00:33<01:17, 1.30it/s]
18%		21/120	[00:34<01:17, 1.28it/s]
18%		22/120	[00:35<01:31, 1.07it/s]
19%		23/120	[00:36<01:15, 1.29it/s]
20%		24/120	[00:37<01:15, 1.27it/s]
21%		25/120	[00:38<01:25, 1.11it/s]
22%		26/120	[00:39<01:24, 1.11it/s]
22%		27/120	[00:40<01:28, 1.05it/s]
23%		28/120	[00:41<01:31, 1.01it/s]
24%		29/120	[00:42<01:23, 1.09it/s]
25%		30/120	[00:43<01:25, 1.05it/s]
26%		31/120	[00:44<01:34, 1.06s/it]
27%		32/120	[00:45<01:20, 1.09it/s]
28%		33/120	[00:45<01:19, 1.09it/s]
28%		34/120	[00:47<01:32, 1.07s/it]
29%		35/120	[00:47<01:17, 1.10it/s]
30%		36/120	[00:48<01:13, 1.15it/s]
31%		37/120	[00:49<01:14, 1.11it/s]
32%		38/120	[00:50<01:12, 1.12it/s]
32%		39/120	[00:51<01:18, 1.04it/s]
33%		40/120	[00:52<01:17, 1.03it/s]
34%		41/120	[00:53<01:12, 1.10it/s]
35%		42/120	[00:54<01:17, 1.00it/s]
36%		43/120	[00:55<01:24, 1.10s/it]
37%		44/120	[00:56<01:09, 1.09it/s]
38%		45/120	[00:57<01:11, 1.05it/s]

38%		46/120	[00:59<01:24,	1.14s/it]
39%		47/120	[00:59<01:08,	1.06it/s]
40%		48/120	[01:00<01:04,	1.12it/s]
41%		49/120	[01:01<01:05,	1.08it/s]
42%		50/120	[01:02<01:06,	1.05it/s]
42%		51/120	[01:03<01:09,	1.01s/it]
43%		52/120	[01:04<01:07,	1.00it/s]
44%		53/120	[01:05<01:07,	1.00s/it]
45%		54/120	[01:06<01:08,	1.04s/it]
46%		55/120	[01:08<01:18,	1.21s/it]

★ Which model performs best?

`gemma3:4b` performs best in the pairwise LLM-judge evaluation, with 43 wins out of the 120 ordered comparisons. It is followed by `gemma3:1b` with 28 wins, `llama3.2` with 27 wins, and `llama3.2:1b` with 22 wins.

This suggests that the Qwen3:8b judge generally prefers the responses from `gemma3:4b` over the other models for the prompts used in this section.

✓ Part 4.2: Attribute Evaluation (5 Points)

Instead of judging outputs in a pairwise fashion, let's instead implement a judge which takes a prompt and outputs a score on a scale of 1-5 for the following attributes:

- Style - is the output well written and entertaining, without being excessive or over the top in its style?
- Correctness - does the output contain only factually correct information?
- Brevity - is the output short and to the point and does not contain unnecessary slop?

This time you will write the prompt. Include your attribute predictions in the output below.

```
attribute_preds = [
    [
        {'style': None, 'correctness': None, 'brevity': None}
        for _ in models
    ]
    for _ in prompts
]
```

```
def get_attribute_judgement_prompt(prompt, response):
    return f"""
Evaluate the response to the user prompt on three separate attributes.

User prompt:
{prompt}

Response:
{response}

Give each attribute an integer score from 1 to 5.

Style:
1 = poorly written, awkward, or excessively styled
5 = clear, natural, well-written, and appropriately presented

Correctness:
1 = substantially incorrect or misleading
5 = factually correct and directly answers the prompt
```

Brevity:

1 = unnecessarily long, repetitive, or full of irrelevant material

5 = concise while still giving enough information to answer the prompt

Judge each attribute independently.

Return ONLY three integers separated by commas in this exact order:

style,correctness,brevity

Example format:

4,5,3

"".strip()

One judge request for each prompt/model response:

10 prompts × 4 models = 40 requests.

attribute_judge_prompts = []

attribute_locations = []

```
for prompt_idx, original_prompt in enumerate(prompts):
    for model_idx, model in enumerate(models):
        response = outputs_per_model[model][prompt_idx]
```

```
        attribute_judge_prompts.append(
            get_attribute_judgement_prompt(
                original_prompt,
                response
            )
        )
```

```
        attribute_locations.append(
            (prompt_idx, model_idx, model)
        )
```

```
attribute_answers = await run_batch_inference(
    judge_model,
    attribute_judge_prompts,
    temperature=0.0,
    max_tokens=16
)
```

Parse answers such as "4,5,3".

invalid_attribute_answers = []

```
for answer, location in zip(attribute_answers, attribute_locations):
    prompt_idx, model_idx, model = location
```

```
    if answer is None:
        invalid_attribute_answers.append(
            (prompt_idx, model, answer)
        )
        continue
```

```
    pieces = [
        piece.strip()
        for piece in answer.strip().split(",")
    ]
```

```
    try:
        scores = [int(piece) for piece in pieces]
```

```

        valid = (
            len(scores) == 3
            and all(1 <= score <= 5 for score in scores)
        )
    except ValueError:
        valid = False

    if not valid:
        invalid_attribute_answers.append(
            (prompt_idx, model, answer)
        )
        continue

    style_score, correctness_score, brevity_score = scores

    attribute_preds[prompt_idx][model_idx] = {
        "style": style_score,
        "correctness": correctness_score,
        "brevity": brevity_score
    }

save_results(
    attribute_preds,
    "results/attribute_preds.json"
)

print("Invalid attribute judgments:",
      len(invalid_attribute_answers))

if invalid_attribute_answers:
    print("Examples:", invalid_attribute_answers[:5])

# Scratch space: compute each model's average attribute scores.
attribute_summary = []

for model_idx, model in enumerate(models):
    style_scores = []
    correctness_scores = []
    brevity_scores = []

    for prompt_idx in range(len(prompts)):
        scores = attribute_preds[prompt_idx][model_idx]

        if scores["style"] is not None:
            style_scores.append(scores["style"])
            correctness_scores.append(scores["correctness"])
            brevity_scores.append(scores["brevity"])

    attribute_summary.append({
        "Model": model,
        "Style": sum(style_scores) / len(style_scores),
        "Correctness": sum(correctness_scores) / len(correctness_scores),
        "Brevity": sum(brevity_scores) / len(brevity_scores)
    })

attribute_table = pd.DataFrame(attribute_summary)

display(
    attribute_table.sort_values(
        "style",
        ascending=False
    )
)

```

```

        "Correctness",
        ascending=False
    ).reset_index(drop=True)
)

for attribute in ["Style", "Correctness", "Brevity"]:
    winning_row = attribute_table.loc[
        attribute_table[attribute].idxmax()
    ]

    print(
        f"Best {attribute.lower()}:",
        winning_row["Model"],
        f"({winning_row[attribute]:.2f})"
    )

```

100%|██████████| 40/40 [00:43<00:00, 1.09s/it]Invalid attribute judgments: 0

	Model	Style	Correctness	Brevity
0	gemma3:4b	4.0	5.0	3.2
1	llama3.2:1b	4.0	4.7	3.2
2	llama3.2	3.9	4.7	3.1
3	gemma3:1b	3.8	3.9	2.6

Best style: llama3.2:1b (4.00)
 Best correctness: gemma3:4b (5.00)
 Best brevity: llama3.2:1b (3.20)

- ★ Which models win on each attribute?
- ★ How do these results correlate with the head-to-head comparisons?

For style, `gemma3:4b` and `llama3.2:1b` tie for the highest average score at 4.0. For correctness, `gemma3:4b` performs best with a perfect average score of 5.0. For brevity, `gemma3:4b` and `llama3.2:1b` tie again at 3.2.

These results partially agree with the pairwise evaluation from Part 4.1. `gemma3:4b` had the most pairwise wins with 43, and it also receives the highest correctness score while tying for the best style and brevity scores. This suggests that correctness may be an important reason the judge prefers `gemma3:4b` overall.

However, the relationship is not perfect. `llama3.2:1b` ties for the best style and brevity scores, but it had the fewest pairwise wins, with only 22. Also, `gemma3:1b` had 28 pairwise wins, slightly more than `llama3.2`, even though it received the lowest average scores for all three attributes. This shows that pairwise judgments capture an overall preference that is not completely explained by these three individual attributes.

Part 4.3: Human Evaluation and Error Analysis (5 Points)

- ★ How do your preferences and ratings correlate or don't correlate with the LLM-as-a-judge labels?
- ★ What is one additional attribute you might want to evaluate via LLM-as-a-judge so that the evaluation better captures your preferences?

Part 4.3 Human Evaluation and Error Analysis

My preferences only partially agree with the LLM-as-a-judge labels. I generally agree that `gemma3:4b` is one of the stronger models because some of its responses are clearer and more direct. For example, its explanation of BLEU score is concise and correctly focuses on n-gram overlap with reference translations.

However, after manually reading the outputs, I noticed several cases where the judge's ratings seem overly generous. In Part 4.2, `gemma3:4b` received an average correctness score of 5.0, but some of its answers contain questionable or incorrect information. For example, its response about CS 61C describes the course as "Programming Methodology" and focuses heavily on object-oriented programming and software design. Other models also give very different descriptions of the same course, including machine learning, algorithms, and graduate-level computer systems. This disagreement across models is a strong signal that at least some of the answers are unreliable.

I also noticed possible hallucinations in the local recommendation questions. Some responses give many specific restaurant names, addresses, prices, and websites with very high confidence. In a few cases, several supposedly different restaurants are even assigned the same address, which makes those recommendations suspicious. A human reader can notice these inconsistencies more easily than an automatic judge that is mainly evaluating how convincing or well-written an answer sounds.

For this reason, my preferences correlate with the LLM judge more on presentation and fluency than on factual reliability. A response can sound polished and receive a high judge score while still containing unsupported or incorrect details.

One additional attribute I would evaluate is **verifiability**. This would measure whether factual claims, especially specific names, locations, statistics, course information, and recommendations, can actually be supported or checked. I think this would better capture my preferences because I would rather have a cautious and verifiable answer than a confident, polished answer containing fabricated details.

▼ Part 4.4: Prompt and Model Sensitivity (5 Points)

Re-run the head-to-head LLM-as-a-judge evaluation in Part 4.1, but instead re-phrase the prompt.

Include your outputs in the `new_head_to_head_judgements` object.

```
new_head_to_head_judgements = [{str((model_a, model_b))}: None for model_a in models for model_b in models]

def get_new_pairwise_judgement_prompt(prompt, output1, output2):
    return (
        f"Read the user request and the two candidate responses below. "
        f"Decide which response is better overall.\n\n"
        f"User request: {prompt}\n\n"
        f"Response 1: {output1}\n\n"
        f"Response 2: {output2}\n\n"
        f"If Response 1 is better, output 1. "
        f"If Response 2 is better, output 2. "
        f"Return only 1 or 2."
    )

new_judge_prompts = []
new_judge_locations = []

for prompt_idx, original_prompt in enumerate(prompts):
    for model_a in models:
        for model_b in models:
            if model_a == model_b:
                continue
```

```

        output_a = outputs_per_model[model_a][prompt_idx]
        output_b = outputs_per_model[model_b][prompt_idx]

        new_judge_prompts.append(
            get_new_pairwise_judgement_prompt(
                original_prompt,
                output_a,
                output_b
            )
        )

        new_judge_locations.append(
            (prompt_idx, model_a, model_b)
        )

new_judge_answers = await run_batch_inference(
    judge_model,
    new_judge_prompts,
    temperature=0.0,
    max_tokens=8
)

for answer, location in zip(
    new_judge_answers,
    new_judge_locations
):
    prompt_idx, model_a, model_b = location

    cleaned_answer = (
        answer.strip()
        if answer is not None
        else ""
    )

    if cleaned_answer == "1":
        winner = model_a
    elif cleaned_answer == "2":
        winner = model_b
    else:
        winner = None

    new_head_to_head_judgements[prompt_idx][
        str((model_a, model_b))
    ] = winner

save_results(new_head_to_head_judgements, "results/new_head_to_head_judgements.json")

# Check for invalid judge responses.
unresolved = sum(
    winner is None
    for prompt_result in new_head_to_head_judgements
    for winner in prompt_result.values()
)

print("Unresolved judgments:", unresolved)

# Count wins under the rephrased prompt.
new_win_counts = {model: 0 for model in models}

for prompt_result in new_head_to_head_judgements:

```

```

    for winner in prompt_result.values():
        if winner in new_win_counts:
            new_win_counts[winner] += 1

# Recompute the original Part 4.1 counts for comparison.
old_win_counts = {model: 0 for model in models}

for prompt_result in head_to_head_judgements:
    for winner in prompt_result.values():
        if winner in old_win_counts:
            old_win_counts[winner] += 1

comparison_table = pd.DataFrame({
    "Model": models,
    "Original Wins": [
        old_win_counts[model] for model in models
    ],
    "Rephrased-Prompt Wins": [
        new_win_counts[model] for model in models
    ]
})

comparison_table["Change"] = (
    comparison_table["Rephrased-Prompt Wins"]
    - comparison_table["Original Wins"]
)

comparison_table = comparison_table.sort_values(
    "Rephrased-Prompt Wins",
    ascending=False
).reset_index(drop=True)

display(comparison_table)

# Count how many individual pairwise decisions changed.
changed_judgements = 0
total_judgements = 0

for prompt_idx in range(len(prompts)):
    for pair in head_to_head_judgements[prompt_idx]:
        old_winner = head_to_head_judgements[prompt_idx][pair]
        new_winner = new_head_to_head_judgements[prompt_idx][pair]

        if old_winner is not None and new_winner is not None:
            total_judgements += 1

            if old_winner != new_winner:
                changed_judgements += 1

print(
    "Changed judgments:",
    f"{changed_judgements}/{total_judgements}"
)

print(
    "Percent changed:",
    f"{100 * changed_judgements / total_judgements:.1f}%"
)

```

100% | ██████████ | 120/120 [02:25<00:00, 1.215/1t] unresolved judgments: 0

	Model	Original Wins	Rephrased-Prompt Wins	Change	
0	gemma3:4b	43	48	5	 
1	gemma3:1b	28	32	4	
2	llama3.2	27	22	-5	
3	llama3.2:1b	22	18	-4	

Changed judgments: 67/120
Percent changed: 55.8%

Next steps: [New interactive sheet](#)

- ★ How does the evaluation change when you change the prompt?
- ★ How might you adjust your evaluation methodology to account for this variation?

Part 4.4 Prompt Sensitivity

Changing only the wording of the judge prompt had a large effect on the pairwise evaluation. In the original evaluation, `gemma3:4b` had 43 wins, `gemma3:1b` had 28, `llama3.2` had 27, and `llama3.2:1b` had 22. With the rephrased prompt, the counts changed to 48, 32, 22, and 18 respectively.

The overall ranking stayed the same, with `gemma3:4b` still performing best, but 67 out of 120 individual pairwise judgments changed, which is 55.8% of all comparisons. This is a surprisingly large amount of variation considering that the candidate responses themselves did not change. It shows that LLM-as-a-judge evaluation can be quite sensitive to seemingly small changes in how the judging instruction is phrased.

To account for this variation, I would avoid relying on a single judge prompt. Instead, I would evaluate the same model outputs using several differently worded but semantically equivalent judge prompts and aggregate the results, for example by majority vote or by averaging win rates across prompts. I would also report the amount of disagreement across prompt variants as a measure of evaluation uncertainty. For especially important comparisons, I would additionally use multiple judge models or human evaluation to reduce dependence on the preferences of one prompt and one judge model.

✓ Part 5: Frontier of Model Capabilities (optional; 10 bonus points)

In this optional part you will invent and evaluate a new task that you believe modern LLMs might struggle with.

Part 5.1: Invent a new task (5 points)

Brainstorm a task that you think modern models might not be able to do (yet, or ever).

Report:

- ★ A natural language description of the task.
- ★ A few instances of the task, showing inputs and expected outputs.

Part 5.2: Evaluate a model on this task (5 points)

How well do current models perform on this task?

How well do current models perform on this task?

Report:

- ★ Explain how you might evaluate this task. What challenges might arise in evaluation?
- ★ Try evaluating a frontier model (e.g. LLM, VLM) on this task. Which model did you use? How did it perform? Show examples of its outputs. If it performed poorly, what do you think are the main challenges involved in task success that the model can't capture?

Homework Feedback (optional)

- ★ This is a new assignment so we are open to feedback. What was good/bad about this assignment? What should we have done differently?

✓ Academic integrity

You can discuss high-level solutions with classmates, but your submitted work should be your own. Don't copy from or share code with one another. Provide attribution for any sources of assistance you used in the assignment. If you use generative AI tools to aid in any parts of this assignment, describe how you used them and what worked (or didn't work) when using them.

I used Chatgpt to explain me the question and tips for the answer, ALL of these work are my work

```
!cd /content/repo && python3 make_submission.py
```

part	file	status
1.2	accuracies.json	ok
1.3	best_model_prec.json	ok
1.4	best_model_rec.json	ok
1.5	f1s.json	ok
2.1	language_wers.json	ok
2.2	language_wer_cis.json	ok
3.1	ngram_precisions_cs.json	ok
3.2	brevity_penalties_cs.json	ok
3.3	bleu_scores_cs.json	ok
3.3	bleu_scores_zh.json	ok
3.3	bleu_scores_gu.json	ok
4.1	head_to_head_judgements.json	ok
4.2	attribute_preds.json	ok
4.4	new_head_to_head_judgements.json	ok

HW1.pdf not found -- export the notebook to PDF for the wr

wrote /content/repo/submission.zip (9 KB, 14 files)

Everything checks out. Upload this to Gradescope.

